



PART 10

APPENDIX A

Appendix A presents the full details of the university database that we have used as our running example, including an E-R diagram, SQL DDL, and sample data that we have used throughout the book. (The DDL and sample data are also available on the web site of the book, db-book.com, for use in laboratory exercises.)

APPENDIX A



Detailed University Schema

In this appendix, we present the full details of our running-example university database. In Section A.1 we present the full schema as used in the text and the E-R diagram that corresponds to that schema. In Section A.2 we present a relatively complete SQL data definition for our running university example. Besides listing a datatype for each attribute, we include a substantial number of constraints. Finally, in Section A.3, we present sample data that correspond to our schema. SQL scripts to create all the relations in the schema, and to populate them with sample data, are available on the web site of the book, db-book.com.

A.1 Full Schema

The full schema of the university database that is used in the text follows. The corresponding schema diagram, and the one used throughout the text, is shown in Figure A.1.

```
classroom(building, room_number, capacity)
department(dept_name, building, budget)
course(course_id, title, dept_name, credits)
instructor(ID, name, dept_name, salary)
section(course_id, sec_id, semester, year, building, room_number, time_slot_id)
teaches(ID, course_id, sec_id, semester, year)
student(ID, name, dept_name, tot_cred)
takes(ID, course_id, sec_id, semester, year, grade)
advisor(s_ID, i_ID)
time_slot(time_slot_id, day, start_time, end_time)
prereq(course_id, prereq_id)
```



```

create table course
  (course_id      varchar (7),
   title          varchar (50),
   dept_name      varchar (20),
   credits         numeric (2,0) check (credits > 0),
   primary key (course_id),
   foreign key (dept_name) references department
     on delete set null);

create table instructor
  (ID            varchar (5),
   name          varchar (20) not null,
   dept_name      varchar (20),
   salary         numeric (8,2) check (salary > 29000),
   primary key (ID),
   foreign key (dept_name) references department
     on delete set null);

create table section
  (course_id      varchar (8),
   sec_id         varchar (8),
   semester       varchar (6) check (semester in
     ('Fall', 'Winter', 'Spring', 'Summer')),
   year           numeric (4,0) check (year > 1701 and year < 2100),
   building       varchar (15),
   room_number    varchar (7),
   time_slot_id   varchar (4),
   primary key (course_id, sec_id, semester, year),
   foreign key (course_id) references course
     on delete cascade,
   foreign key (building, room_number) references classroom
     on delete set null);

```

In the preceding DDL, we add the **on delete cascade** specification to a foreign key constraint if the existence of the tuple depends on the referenced tuple. For example, we add the **on delete cascade** specification to the foreign key constraint from *section* (which was generated from weak entity *section*), to *course* (which was its identifying relationship). In other foreign key constraints we either specify **on delete set null**, which allows deletion of a referenced tuple by setting the referencing value to null, or we do not add any specification, which prevents the deletion of any referenced tuple. For example, if a department is deleted, we would not wish to delete associated instructors;

the foreign key constraint from *instructor* to *department* instead sets the *dept_name* attribute to null. On the other hand, the foreign key constraint for the *prereq* relation, shown later, prevents the deletion of a course that is required as a prerequisite for another course. For the *advisor* relation, shown later, we allow *i_ID* to be set to null if an instructor is deleted but delete an *advisor* tuple if the referenced student is deleted.

create table *teaches*

```
(ID          varchar (5),
 course_id   varchar (8),
 sec_id      varchar (8),
 semester    varchar (6),
 year        numeric (4,0),
 primary key (ID, course_id, sec_id, semester, year),
 foreign key (course_id, sec_id, semester, year) references section
               on delete cascade,
 foreign key (ID) references instructor
               on delete cascade);
```

create table *student*

```
(ID          varchar (5),
 name        varchar (20) not null,
 dept_name    varchar (20),
 tot_cred     numeric (3,0) check (tot_cred >= 0),
 primary key (ID),
 foreign key (dept_name) references department
               on delete set null);
```

create table *takes*

```
(ID          varchar (5),
 course_id   varchar (8),
 sec_id      varchar (8),
 semester    varchar (6),
 year        numeric (4,0),
 grade       varchar (2),
 primary key (ID, course_id, sec_id, semester, year),
 foreign key (course_id, sec_id, semester, year) references section
               on delete cascade,
 foreign key (ID) references student
               on delete cascade);
```

```

create table advisor
  (s_ID          varchar (5),
   i_ID          varchar (5),
   primary key (s_ID),
   foreign key (i_ID) references instructor (ID)
                        on delete set null,
   foreign key (s_ID) references student (ID)
                        on delete cascade);

```

```

create table prereq
  (course_id    varchar(8),
   prereq_id    varchar(8),
   primary key (course_id, prereq_id),
   foreign key (course_id) references course
                        on delete cascade,
   foreign key (prereq_id) references course);

```

The following **create table** statement for the table *time_slot* can be run on most database systems, but it does not work on Oracle (at least as of Oracle version 11), since Oracle does not support the SQL standard type **time**.

```

create table timeslot
  (time_slot_id varchar (4),
   day          varchar (1) check (day in ('M', 'T', 'W', 'R', 'F', 'S', 'U')),
   start_time   time,
   end_time     time,
   primary key (time_slot_id, day, start_time));

```

The syntax for specifying time in SQL is illustrated by these examples: '08:30', '13:55', and '5:30 PM'. Since Oracle does not support the **time** type, for Oracle we use the following schema instead:

```

create table timeslot
  (time_slot_id varchar (4),
   day          varchar (1),
   start_hr     numeric (2) check (start_hr >= 0 and end_hr < 24),
   start_min    numeric (2) check (start_min >= 0 and start_min < 60),
   end_hr       numeric (2) check (end_hr >= 0 and end_hr < 24),
   end_min      numeric (2) check (end_min >= 0 and end_min < 60),
   primary key (time_slot_id, day, start_hr, start_min));

```

The difference is that *start_time* has been replaced by two attributes *start_hr* and *start_min*, and similarly *end_time* has been replaced by attributes *end_hr* and *end_min*.

These attributes also have constraints that ensure that only numbers representing valid time values appear in those attributes. This version of the schema for *time_slot* works on all databases, including Oracle. Note that although Oracle supports the **datetime** datatype, **datetime** includes a specific day, month, and year as well as a time, and is not appropriate here since we want only a time. There are two alternatives to splitting the time attributes into an hour and a minute component, but neither is desirable. The first alternative is to use a **varchar** type, but that makes it hard to enforce validity constraints on the string as well as to perform comparison on time. The second alternative is to encode time as an integer representing a number of minutes (or seconds) from midnight, but this alternative requires extra code with each query to convert values between the standard time representation and the integer encoding. We therefore choose the two-part solution.

A.3 Sample Data

In this section we provide sample data for each of the relations defined in the previous section.

<i>building</i>	<i>room_number</i>	<i>capacity</i>
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

Figure A.2 The *classroom* relation.

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Figure A.3 The *department* relation.

Credits

The photo of the sailboats in the beginning of the chapter is due to ©Pavel Nesvadba/Shutterstock.

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure A.4 The *course* relation.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure A.5 The *instructor* relation.

<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>room_number</i>	<i>time_slot_id</i>
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A

Figure A.6 The *section* relation.

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	CS-101	1	Fall	2017
10101	CS-315	1	Spring	2018
10101	CS-347	1	Fall	2017
12121	FIN-201	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017
32343	HIS-351	1	Spring	2018
45565	CS-101	1	Spring	2018
45565	CS-319	1	Spring	2018
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
83821	CS-319	2	Spring	2018
98345	EE-181	1	Spring	2017

Figure A.7 The *teaches* relation.

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>tot_cred</i>
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

Figure A.8 The *student* relation.

<i>ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>grade</i>
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	<i>null</i>

Figure A.9 The *takes* relation.

<i>s_id</i>	<i>i_id</i>
00128	45565
12345	10101
23121	76543
44553	22222
45678	22222
76543	45565
76653	98345
98765	98345
98988	76766

Figure A.10 The *advisor* relation.

<i>time_slot_id</i>	<i>day</i>	<i>start_time</i>	<i>end_time</i>
A	M	8:00	8:50
A	W	8:00	8:50
A	F	8:00	8:50
B	M	9:00	9:50
B	W	9:00	9:50
B	F	9:00	9:50
C	M	11:00	11:50
C	W	11:00	11:50
C	F	11:00	11:50
D	M	13:00	13:50
D	W	13:00	13:50
D	F	13:00	13:50
E	T	10:30	11:45
E	R	10:30	11:45
F	T	14:30	15:45
F	R	14:30	15:45
G	M	16:00	16:50
G	W	16:00	16:50
G	F	16:00	16:50
H	W	10:00	12:30

Figure A.11 The *time_slot* relation.

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Figure A.12 The *prereq* relation.

<i>time_slot_id</i>	<i>day</i>	<i>start_hr</i>	<i>start_min</i>	<i>end_hr</i>	<i>end_min</i>
A	M	8	0	8	50
A	W	8	0	8	50
A	F	8	0	8	50
B	M	9	0	9	50
B	W	9	0	9	50
B	F	9	0	9	50
C	M	11	0	11	50
C	W	11	0	11	50
C	F	11	0	11	50
D	M	13	0	13	50
D	W	13	0	13	50
D	F	13	0	13	50
E	T	10	30	11	45
E	R	10	30	11	45
F	T	14	30	15	45
F	R	14	30	15	45
G	M	16	0	16	50
G	W	16	0	16	50
G	F	16	0	16	50
H	W	10	0	12	30

Figure A.13 The *time_slot* relation with start and end times separated into hour and minute.

Index

- aborted transactions, 805–807, 819–820**
- abstraction, 2, 9–12, 15**
- acceptors, 1148, 1152**
- accessing data. *See also* security**
 - from application programs, 16–17
 - concurrent-access anomalies, 7
 - difficulties in, 6
 - indices for, 19
 - recovery systems and, 910–912
 - types of access, 15
- access paths, 695**
- access time**
 - indices and, 624, 627–628
 - query processing and, 692
 - storage and, 561, 566, 567, 578
- access types, 624**
- account nonces, 1271**
- ACID properties. *See* atomicity; consistency; durability; isolation**
- Active Server Page (ASP), 405**
- active transactions, 806**
- ActiveX DataObjects (ADO), 1239**
- adaptive lock granularity, 969–970**
- add constraint, 146**
- ADO (ActiveX DataObjects), 1239**
- ADO.NET, 184, 1239**
- Advanced Encryption Standard (AES), 448, 449**
- advanced SQL, 183–231**
 - accessing from programming languages, 183–198
 - aggregate features, 219–231
 - embedded, 197–198
 - functions and procedures, 198–206
 - JDBC and, 184–193
 - ODBC and, 194–197
 - Python and, 193–194
 - triggers and, 206–213
- advertisement data, 469**
- AES (Advanced Encryption Standard), 448, 449**
- after triggers, 210**
- aggregate functions, 91–96**
 - basic, 91–92
 - with Boolean values, 96
 - defined, 91
 - with grouping, 92–95
 - having clause, 95–96
 - with null values, 96
- aggregation**
 - defined, 277
 - entity-relationship (E-R) model and, 276–277
 - intraoperation parallelism and, 1049
 - on multidimensional data, 527–532
 - partial, 1049
 - pivoting and, 226–227, 530
 - query optimization and, 764
- query processing and, 723**
- ranking and, 219–223**
- representation of, 279**
- rollup and cube, 227–231**
- skew and, 1049–1050**
- of transactions, 1278**
- view maintenance and, 781–782**
- windowing and, 223–226**
- aggregation operation, 57**
- aggregation switch, 977**
- airlines, database applications for, 3**
- Ajax, 423–426, 1015**
- algebraic operations. *See* relational algebra**
- aliases, 81, 336, 1242**
- all construct, 100**
- alter table, 71, 146**
- alter trigger, 210**
- alter type, 159**
- Amdahl’s law, 974**
- American National Standards Institute (ANSI), 65, 1237**
- analysis pass, 944**
- analytics. *See* data analytics**
- and connective, 74**
- and operation, 89–90**
- anonymity, 1252, 1253, 1258, 1259**
- ANSI (American National Standards Institute), 65, 1237**
- anticipatory standards, 1237**
- anti-join operation, 108, 776**

anti-semijoin operation, 776–777**Apache**

- AsterixDB, 668
- Cassandra, 477, 489, 668, 1024, 1028
- Flink project, 504, 508
- Giraph system, 511
- HBase, 477, 480, 489, 668, 971, 1024, 1028–1031
- Hive, 494, 495, 500
- Jakarta Project, 416
- Kafka system, 506, 507, 1072–1073, 1075, 1137
- Spark, 495–500, 508, 511, 1061
- Storm stream-processing system, 506–508
- Tez, 495

APIs. *See* application program interfaces**application design, 403–453**

- application architectures and, 429–434
- authentication and, 441–443
- business logic and, 23, 404, 411–412, 430, 431, 445
- client-server architecture and, 404
- client-side code and web services, 421–429
- common gateway interface standard and, 409
- cookies and, 410–415, 411n2
- data access layer and, 430–434
- disconnected operation and, 427–428
- encryption and, 447–453
- HTML and, 404, 406–408, 426
- HTTP and, 405–413
- JavaScript and, 404–405, 421–426
- Java Server Pages and, 405, 417–418
- mobile application platforms, 428–429
- performance and, 434–437
- security and, 437–446
- servlets and, 411–421

- standardization and, 1237–1240
- testing, 1234–1235
- tuning and (*see* performance tuning)
- URLs and, 405–406
- user interfaces and, 403–405
- web and, 405–411

application migration,**1035–1036****application program interfaces****(APIs)**

- ADO, 1239
- ADO.NET, 184, 1239
- application design and, 411, 413, 416
- C++, 1239
- database access from, 16–17
- Java (*see* Java)
- LDAP, 1243
- map displays and, 393
- MongoDB, 477–479, 482, 489, 668, 1024, 1028
- Open Database Connectivity (*see* ODBC)
- Python (*see* Python)
- Spark, 495–500, 508, 511, 1061
- standards for, 1238–1240
- system architectures and, 962
- Tez, 495
- web services and, 427

application programmers, 24**application servers, 23, 416****architectures, 961–995**

- business logic and, 23
- centralized databases, 962–963
- client-server systems, 23, 971
- cloud-based, 990–995, 1026, 1027
- database storage, 587–588
- data-server systems, 963–964, 968–970
- data warehousing, 522–523
- destination-driven, 522
- distributed databases, 22, 986–989, 1098–1100
- hierarchical, 979, 980, 986
- hypercube, 976–977

- lambda, 504, 1071
- mesh, 976
- microservices, 994
- multiuser systems, 962
- Non-Uniform Memory Access, 981, 1063
- overview, 961–962
- parallel databases, 22, 970–986
- platform-as-a-service model, 992–993
- recovery systems, 932
- server system, 962–970, 977–978
- shared disk, 979, 980, 984–985
- shared memory, 21–22, 979–984, 1061–1064
- shared nothing, 979, 980, 985–986, 1040–1041, 1061–1063
- single-user systems, 962
- software-as-a-service model, 993
- source-driven, 522
- storage area network, 562
- three-tier, 23
- transaction-server systems, 963–968
- two-phase commit protocol, 989, 1276
- two-tier, 23
- wide-area networks, 989

archival data, 561**archival dump, 931****ARIES**

- analysis pass and, 944
- compensation log records and, 942, 945
- data structures and, 942–944
- dirty page table and, 941–947
- fine-grained locking and, 947
- fuzzy checkpoints and, 941
- log sequence number and, 941–946
- nested top actions and, 946
- optimization and, 947
- physiological redo and, 941
- recovery algorithm, 944–946, 1276

- redo pass and, 944–945
- rollback and, 945–946
- savepoints and, 947
- undo pass and, 944–946
- arity, 54**
- Armstrong's axioms, 321**
- array databases, 367**
- array types, 366, 367, 378**
- asc expression, 84**
- as clause, 79, 81**
- as of period for, 157**
- ASP (Active Server Page), 405**
- ASP.NET, 417**
- assertions, 152–153**
- asset management, 1277**
- assignment operation, 55–56, 201**
- associations**
 - data mining and, 541
 - entity sets (*see* entity sets)
 - relation schema for, 42
 - relationship sets (*see* relationship sets)
 - rules for, 546–547
- associative property, 749–750**
- AsterixDB, 668**
- asymmetric**
 - fragment-and-replicate joins, 1046, 1062**
- asymmetric-key encryption, 448**
- asynchronous replication, 1122, 1135–1138**
- asynchronous view maintenance, 1138–1140**
- at-least-once semantics, 1074**
- at-most-once semantics, 1074**
- atomic commit, 1029**
- atomic domains, 40, 342–343**
- atomic instructions, 966–967**
- atomicity**
 - cascadeless schedules and, 820–821
 - commit protocols and, 1100–1110
 - defined, 20, 800
 - in file-processing systems, 6–7
 - isolation and, 819–821
 - log records and, 913–919
 - recoverable schedules and, 819–820
- recovery systems and, 803, 912–922
- storage structure and, 804–805
- of transactions, 20–21, 144, 481, 800–807, 819–821
- attribute inheritance, 274–275**
- attributes**
 - atomic domains and, 342–343
 - bitmap indices and, 670–672
 - classifiers and, 541–543, 545
 - closure of attribute sets, 322–324
 - complex, 249–252, 265–267
 - composite, 250, 252
 - decomposition and, 305–313, 330–335, 339–340
 - derived, 251, 252
 - descriptive, 248
 - design issues and, 281–282
 - discriminator, 259
 - domain of, 39–40, 249
 - entity-relationship diagrams and, 265–267
 - entity-relationship (E-R)
 - model and, 245, 248–252, 274–275, 281–282, 342–343
 - entity sets and, 245, 265–267, 281–282
 - extraneous, 325
 - histograms and, 758–760
 - multiple-key access and, 661–663
 - multivalued, 251, 252, 342
 - naming of, 345–346
 - null values and, 251–252
 - partitioning, 479
 - primary keys and, 310n4
 - prime, 356
 - in relational model, 39
 - relationship sets and, 248
 - search key and, 624
 - simple, 250, 265
 - single-valued, 251
 - Unified Modeling Language and, 289
 - uniquifiers, 649–650
 - value set of, 249
- attribute-value skew, 1008**
- audit trails, 445–446**
- augmentation rule, 321**
- authentication**
 - application-level, 441–443
 - challenge-response system and, 451
 - digital certificates and, 451–453
 - digital signatures and, 451
 - encryption and, 450–453
 - single sign-on system and, 442–443
 - smart cards and, 451, 451n9
 - two-factor, 441–442
 - web sessions and, 410
- authorization**
 - administrators and, 166
 - application-level, 443–445
 - database design and, 291
 - end-user information and, 443
 - granting privileges, 25, 166–167, 170–171
 - lack of fine-grained, 443–445
 - permissioned blockchains and, 1253
 - revoking privileges, 166–167, 171–173
 - roles and, 167–169
 - row-level, 173
 - on schema, 170
 - Security Assertion Markup Language and, 442–443
 - SQL DDL and, 66
 - sql security invoker, 170
 - storage manager and, 19
 - transfer of privileges, 170–171
 - types of, 14, 165
 - updates and, 14, 170, 171
 - on views, 169–170
- authorization graph, 171**
- automatic commit, 144, 144n6, 822**
- autonomous smart contracts, 1272–1273**
- autonomy, 988**
- availability**
 - CAP theorem and, 1134
 - distributed databases and, 987

- high availability, 907, 931–933, 987, 1121
- network partitions and, 481, 989
- robustness and, 1121
- trading off consistency for, 1134–1135
- average latency time, 566**
- average response time, 809**
- average seek time, 566**
- avg expression, 91–96, 723, 781–782**
- Avro, 490, 499**
- axioms, 321**
- Azure Stream Analytics, 505**
- backpropagation algorithm, 545**
- backup. *See also* recovery systems**
 - application design and, 450
 - remote systems for, 909, 931–935
 - replica systems, 212–213
 - transactions and, 805
- backup coordinators, 1146–1147**
- balanced range-partitioning vector, 1008–1009**
- balanced trees, 634**
- banking**
 - analytics for, 520–521
 - database applications for, 3, 4, 7, 144
- BASE properties, 1135**
- base query, 217**
- batch scaleup, 973**
- batch update, 1221**
- Bayesian classifiers, 543–544**
- Bayes' theorem, 543**
- BCNF. *See* Boyce–Codd normal form**
- BCNF decomposition algorithm, 331–333, 336**
- before triggers, 210**
- begin atomic...end, 145, 201, 208, 209, 211**
- begin transaction operation, 799**
- benchmarks. *See* performance benchmarks**
- bestplan array, 768–770**
- biased protocol, 1124**
- BI (business intelligence), 521**
- big-bang approach, 1236**
- BigchainDB, 1269**
- Big Data, 467–511**
 - algebraic operations and, 494–500
 - comparison with traditional databases, 468
 - defined, 467
 - distributed file systems for, 472–475, 489
 - graph databases and, 508–511
 - key-value systems and, 471, 473, 476–480
 - MapReduce paradigm and, 481, 483–494
 - motivations for, 467–472
 - parallel and distributed databases for, 473, 480–481
 - query processing and, 470–472
 - replication and consistency in, 481–482
 - sharding and, 473, 475–476
 - sources and uses of, 468–470
 - storage systems, 472–482, 668
 - streaming data, 468, 500–508
- Bigtable, 477, 479–480, 668, 1024–1025, 1028–1030**
- binary operations, 48**
- binary relationship sets, 249, 283–285**
- Bing Maps, 393**
- Bitcoin**
 - anonymity and, 1253, 1258
 - data mining and, 1265
 - forking and, 1258
 - growth and development of, 1251–1253
 - language and, 1269–1270
 - processing speed, 1274
 - as public blockchain, 1253, 1255, 1263
 - transactions and, 1261–1263, 1268–1271
- bit-level striping, 571–572**
- bitmap indices**
 - attributes and, 670–672
 - B+-trees and, 1185–1186
- efficient implementation of, 1184–1185
- existence, 1184
- intersection and, 671, 1183
- processing speed and, 662, 663
- scans of, 698–699
- sequential records and, 1182
- structure of, 1182–1184
- usefulness of, 671–672
- bit rot, 575**
- blind writes, 868**
- B-link trees, 886**
- blobs, 156, 193, 594, 652**
- blockchain databases, 1251–1279**
 - anonymity and, 1252, 1253, 1258, 1259
 - applications for use, 1251–1252, 1276–1279
 - concurrency control and, 1262–1263
 - consensus mechanisms for, 1254, 1256–1257, 1263–1267
 - cryptocurrencies and, 1251–1253, 1257
 - cryptographic hash functions and, 1253, 1259–1263, 1265
 - data mining and, 1256, 1258, 1264–1266
 - decentralization and, 1251, 1252, 1259, 1270
 - digital ledgers and, 1251, 1252
 - digital signatures and, 1257, 1261
 - encryption and, 1260–1261
 - external input and, 1271–1272
 - fault-tolerance of, 1276
 - forking and, 1257, 1258, 1263
 - genesis blocks and, 1254–1255
 - irrefutability and, 1257, 1259
 - languages and, 1258, 1269–1271
 - lookup in, 1268
 - management of data in, 1267–1269

- orphaned blocks and, 1257, 1263
- performance enhancement of, 1274–1276
- permissioned, 1253–1254, 1256–1257, 1263, 1266, 1274
- properties and components of, 1254–1259, 1274
- public, 1253, 1255, 1257–1259, 1263, 1264
- query processing and, 1254, 1275–1276
- scalability of, 1276
- smart contracts and, 1258, 1269–1273
- state-based, 1269, 1271
- tamper resistant nature of, 1253–1255, 1259, 1260
- transactions and, 1261–1263, 1268–1271, 1273
- block identifiers, 474–475, 1020**
- blocking edges, 728**
- blocking factor, 725**
- blocking operations, 728, 728n7**
- blocking problem, 1104–1106**
- block-interleaved parity organization, 573**
- block-level striping, 572**
- block nested-loop join, 705–707**
- block-oriented interface, 560**
- blocks**
 - buffer, 910–912
 - dirty, 928–929
 - disk, 566–567, 577–580
 - evicted, 605
 - file organization and, 588
 - genesis, 1254–1255
 - orphaned, 1257, 1263
 - overflow, 598
 - physical, 910
 - pinned, 605
- Bloom filters, 667, 1083, 1175–1176, 1181**
- Boolean operations, 89, 96, 103, 188, 201, 1242. See also *specific operations***
- bottlenecks**
 - application design and, 437
 - I/O parallelism and, 1007
 - performance tuning and, 1211–1213, 1215, 1227
 - single lock-manager and, 1111
 - system architectures and, 981, 985
- bottom-up B⁺-tree construction, 654–655**
- bottom-up design, 273**
- bounding boxes, 674–675, 1187**
- Boyce–Codd normal form (BCNF)**
 - comparison with third normal form, 318–319
 - decomposition algorithm and, 331–333, 336
 - defined, 313–315
 - dependency preservation and, 315–316
 - relational database design and, 313–316
 - testing for, 330–331
- broadcasting, 1055–1056**
- broadcast join, 1046**
- BSP (bulk synchronous processing), 510–511**
- B-trees, comparison with B⁺-trees, 655–656**
- B⁺-trees, 634–658**
 - balanced, 634
 - bitmap indices and, 1185–1186
 - bottom-up construction of, 654–655
 - bulk loading of, 653–655
 - comparison with B-trees, 655–656
 - deletion and, 641, 645–649
 - extensions and variations of, 650–658
 - fanout and, 635
 - on flash storage, 656–657
 - indexing strings and, 653
 - insertion and, 641–645, 647, 649
 - internal nodes and, 635
 - leaf nodes of, 635–656, 665–669, 673, 674
 - in main memory, 657–658
 - nonleaf nodes of, 635–636, 642, 645–656, 663
- nonunique search keys and, 649–650
- organization of, 595, 650–652, 697, 697n4
- parallel key-value stores and, 1028
- performance and, 634, 665–666
- queries on, 637–641, 690
- record relocation and, 652–653
- secondary indices and, 652–653
- spatial data and, 672–673
- structure of, 634–637
- temporal data and, 676
- tuning of, 1215
- updates on, 641–649
- buckets, 659–661, 1194–1195**
- buffer blocks, 910–912**
- buffer manager, 19, 604–607**
- buffers**
 - database buffering, 927–929
 - defined, 604
 - disk blocks and, 578, 910
 - double buffering, 725
 - force/no-force policy and, 927
 - force output and, 912
 - log-record, 926–927
 - management of, 926–930
 - operating system role in, 929–930
 - output of blocks and, 606–607
 - recovery systems and, 926–930
 - reordering of writes and recovery, 609–610
 - replacement strategies, 605, 607–609
 - shared and exclusive locks on, 605–606
 - steal/no-steal policy and, 927
 - storage and, 604–610
 - transaction servers and, 965
 - write-ahead logging rule and, 926–929
- buffer trees, 668–670**
- bugs**

- application design and, 440, 1234, 1236
 - debugging, 199n4
 - failure classification and, 908
- build input, 713**
- bulk export utility, 1222**
- bulk insert utility, 1222**
- bulk loads, 653–655, 1221–1223**
- bulk synchronous processing (BSP), 510–511**
- bully algorithm, 1148**
- business intelligence (BI), 521**
- business logic, 23, 198, 404, 411–412, 430–431, 445**
- business-logic layer, 430, 431**
- business rules, 431**
- bus system, 975–976**
- Byzantine consensus, 1254, 1256, 1266–1267, 1276**
- Byzantine failure, 1266–1267**
- C**
 - advanced SQL and, 183, 197, 199, 205
 - application design and, 16
 - ODBC and, 195–196
 - struct declarations used by, 11n1
 - Unified Modeling Language and, 289
- C++**
 - advanced SQL and, 197, 199, 205, 206
 - application design and, 16, 417
 - object-oriented programming and, 377
 - standards for, 1239
 - struct declarations used by, 11n1
 - Unified Modeling Language and, 289
- cache-conscious algorithms, 732–733**
- cache line, 732, 983**
- cache memory, 559**
- cache misses, 982**
- caching**
 - application design and, 435–437
 - coherency and, 969, 983–984
 - column-oriented storage and, 612
 - data servers and, 968–970
 - locks and, 969
 - query plans and, 774, 965
 - replication and, 1014n4
 - shared-memory architecture and, 982–984
- CAD (computer-aided design), 390–391, 968**
- callable statements, 190–191**
- call back, 969**
- Call Level Interface (CLI)**
 - standards, 197, 1238–1239
- call statement, 201**
- candidate keys, 44**
- canonical cover, 324–328**
- CAP theorem, 1134**
- Cartesian-product operation, 50–52**
- Cartesian products**
 - equivalence and, 748, 749, 755
 - join expressions and, 135
 - query optimization and, 748, 749, 755, 763–764, 775
 - SQL and, 76–79, 81, 127n1, 230
- cascadeless schedules, 820–821**
- cascades, 150, 172, 210**
- cascading rollback, 820–821, 841–842**
- cascading stylesheet (CSS)**
 - standard, 408
- case construct, 112–113**
- Cassandra, 477, 489, 668, 1024, 1028**
- cast, 155, 159**
- catalogs**
 - application design and, 1239
 - indices and (*see* indices)
 - query optimization and, 758–760, 762, 764
 - SQL and, 162–163, 192, 196–197
 - system, 602–604, 1009
- centralized databases, 962–963**
- centralized deadlock detection, 1114**
- centroid, 548, 548n3**
- CEP (complex event processing) systems, 504**
- CGI (common gateway interface) standard, 409**
- chain replication protocol, 1127–1128**
- challenge-response system, 451**
- change isolation level, 822**
- change relation, 211**
- char, 67**
- check clause**
 - assertions and, 152–153
 - integrity constraints and, 147–149, 152–153
 - user-defined types and, 159
- check constraints, 151, 170, 315, 800**
- checkpoint log records, 943**
- checkpoint process, 965**
- checkpoints**
 - fuzzy, 922, 930, 941
 - recovery systems and, 920–922, 930
 - transaction servers and, 965
- checksums, 565**
- chicken-little approach, 1236**
- Chubby, 1150**
- circular arcs, 388**
- classifiers**
 - attributes and, 541–543, 545
 - Bayesian, 543–544
 - data mining and, 541–546
 - decision-tree, 542
 - neural-net, 545–546
 - prediction and, 541–543, 545–546
 - Support Vector Machine, 544–545
 - training instances and, 541
- CLI (Call Level Interface)**
 - standards, 197, 1238–1239
- click-through, 469**
- client-server systems**
 - application design and, 404, 1221, 1239
 - recovery systems and, 931
 - system architecture and, 23, 971

- client-side scripting languages, 421–429
- clobs, 156, 193, 594, 652
- closed addressing, 659, 1194
- closed hashing, 659, 1194
- closed polygons, 388n3
- closed time intervals, 675
- closure of a set, 312, 320–324
- cloud-based data storage, 28, 563, 992–993
- cloud computing
 - architecture for, 990–995, 1026, 1027
 - benefits and limitations of, 995
 - service models, 991–995
 - storage systems and, 28, 563
- CLR (Common Language Runtime), 206
- CLRs (compensation log records), 922, 942, 945
- clustering indices, 625, 632–633, 695, 697–698
- cluster key, 600–601
- cluster membership, 1158
- clusters
 - data mining and, 541, 548–549
 - hierarchical, 548
 - key-value storage systems and, 477
 - multitable, 595, 598–601
 - system architecture and, 978
- coalesce function, 114, 155, 230–231
- coalescing nodes, 641, 886
- coarse-grained parallelism, 963, 970
- Codd, Edgar, 26
- code breaking. *See* encryption
- collision resistant hash functions, 1259–1260
- colocation of data, 1068–1069
- column family, 1025
- column-oriented storage, 525–526, 588, 611–617, 734, 1182
- column stores, 612, 615, 1025, 1224
- combinatorics, 811
- combine function, 490
- comma-separated values, 1222
- commit dependency, 847
- commit protocols, 1100–1110
- committed transactions
 - defined, 806
 - durability and, 933–934
 - log records and, 917
 - observable external writes and, 807
 - partially committed, 806
 - scheduling and, 810, 819–820
 - updates and, 874
- commit time, 933–934
- commit wait, 1130–1131
- commit work, 143–145
- common gateway interface (CGI) standard, 409
- Common Language Runtime (CLR), 206
- common subexpression elimination, 785
- commutative property, 747–750
- commute, 1143
- compare-and-swap, 966
- compatibility function, 836
- compatible relations, 54
- compensating operation, 892
- compensating transactions, 805
- compensation log records (CLRs), 922, 942, 945
- complete axioms, 321
- completeness constraint, 275
- complex attributes, 249–252, 265–267
- complex data types, 365–394
 - object orientation, 376–382
 - semi-structured, 365–376
 - spatial, 387–394
 - textual, 382–387
 - user-defined, 158
- complex event processing (CEP) systems, 504
- composite attributes, 250, 252
- composite indices, 700
- composite price/performance metric, 1234
- composite query per hour metric, 1234
- compression
 - column-oriented storage and, 611, 612
 - data warehousing and, 526
 - of disk block data, 615n8
 - prefix, 653
 - workload compression, 1217
- computer-aided design (CAD), 390–391, 968
- conceptual-design phase, 17–18, 242
- concurrency control, 835–894
 - access anomalies and, 7
 - blind writes and, 868
 - blockchain databases and, 1262–1263
 - commit protocols and, 1105
 - consistency and, 880–885
 - deadlock handling and, 849–853
 - deletion and, 857–858
 - distributed databases and, 990, 1105, 1111–1120
 - extended protocols, 1129–1133
 - false cycles and, 1114–1115
 - in federated databases, 1132–1133
 - indices and, 884–887
 - insertion and, 857, 858
 - isolation and, 803–804, 807–812, 823
 - leases and, 1115–1116
 - locking protocols and, 835–848 (*see also* locks)
 - logical undo operations and, 940–941
 - long-duration transactions and, 890–891
 - in main-memory databases, 887–890
 - multiple granularity and, 853–857
 - multiversion schemes and, 869–872, 1129–1131
 - with operations, 891–894
 - optimistic, 869
 - parallel databases and, 990
 - parallel key-value stores and, 1028–1029

- phantom phenomenon and, 827, 858–861, 877–879, 877n5, 885, 887
- predicate reads and, 858–861
- real-time transaction systems and, 894
- recovery systems and, 916
- replication and, 1123–1125
- rollback and, 841–844, 849–850, 853, 868–871
- serializability and, 836, 840–843, 846–848, 856, 861–871, 875–887
- snapshot isolation and, 872–879, 882, 916, 1131–1132
- timestamp-based protocols and, 861–866, 882
- trends in, 808
- user interactions and, 881–883
- validation and, 866–869, 882, 916
- concurrency-control manager, 21**
- concurrency-control schemes, 809**
- concurrent transactions, 1224–1227**
- confidence, 540, 547**
- conflict equivalence, 815, 815n2**
- conflict serializability, 813–816**
- conformance levels, 196–197**
- conjunctive selection, 699–700, 747, 762**
- connection pooling, 436**
- consensus protocols**
 - blockchain databases and, 1254, 1256–1257, 1263–1267
 - Byzantine, 1254, 1256, 1266–1267, 1276
 - distributed databases and, 1106–1107, 1150–1161, 1266, 1267
 - message-based, 1266
 - multiple consensus protocol, 1151
 - Paxos, 1152–1155, 1160–1161, 1267
 - proof-of-stake, 1256, 1266
 - proof-of-work, 1256, 1264–1266
 - Raft, 1148, 1155–1158, 1267
 - replication and, 1016
 - Zab, 1152
- consistency**
 - Big Data and, 481–482
 - CAP theorem and, 1134
 - concurrency control and, 880–885
 - cursor stability and, 881
 - deadlock and, 838–839
 - defined, 20
 - degree-two, 880–881
 - eventual, 1016, 1139
 - external, 1131
 - file system consistency check, 610
 - hashing and, 1013
 - logical operations and, 936–937
 - replication and, 1015–1016, 1121–1123, 1133–1146
 - requirement of, 802
 - trading off for availability, 1134–1135
 - of transactions, 20, 800, 802, 807–808, 821–823
 - user interactions and, 881–883
 - weak levels of, 880–883
- constraints**
 - check, 151, 170, 315
 - completeness, 275
 - consistency, 13–14
 - deadlines, 894
 - decomposition and, 336
 - dependency preservation and, 315–316
 - entity-relationship (E-R) model and, 253–256, 275–276
 - foreign key, 45–46
 - integrity (*see* integrity constraints)
 - keys and, 258
 - mapping cardinalities and, 253–256
 - not null, 69
 - primary key, 44
 - on specialization, 275–276
 - transactions and, 800
 - Unified Modeling Language and, 289
- containers, 992–994**
- contains operation, 101**
- continuous queries, 503, 731**
- continuous-stream data, 731**
- conversations, 883**
- conversions, 155–156, 469, 843**
- cookies, 410–415, 411n2, 439–440**
- coordinators, 1099, 1104, 1106–1107, 1146–1150**
- Corda, 1269**
- cores, 962–963, 970, 976, 980–983**
- core switch, 977**
- correlated evaluation, 775**
- correlated subqueries, 101**
- correlation name, 81, 101**
- correlation variables, 81, 775**
- cost-based optimizers, 766**
- Couchbase, 1024**
- count function, 91–92, 94, 96, 723, 766, 781**
- count values, 220n11**
- covering indices, 663**
- crabbing protocol, 885–886**
- crashes. *See also* recovery systems**
 - actions following, 923–925
 - algorithms for, 922–925
 - ARIES and, 941–947
 - checkpoints and, 920–922
 - failure classification and, 908
 - magnetic disks and, 565
 - storage and, 607, 609–610
 - transactions and, 800
- crawling the web, 383**
- create assertion, 153**
- create cluster, 601**
- create distinct type, 160**
- create domain, 159–160**
- create function, 200, 203, 204, 215**
- create index, 164–165, 664**
- create or replace, 199n4**
- create procedure, 200, 205**
- create recursive view, 218**

- create role, 168
 - create schema, 163
 - create sequence construct, 161
 - create table
 - with data clause, 162
 - default values and, 156
 - extensions for, 162
 - integrity constraints and, 146–149
 - multitable clustering and, 601
 - object-based databases and, 378–380
 - shipping SQL statements to database and, 187
 - SQL schema definition and, 68–71
 - create table...as, 162
 - create table...like, 162
 - create temporary table, 214
 - create type, 158–160, 378–380
 - create unique index, 165, 664
 - create view, 138–143, 162, 169
 - credit bureaus, 521, 521n1
 - cross-chain transactions, 1273
 - cross join, 127n1
 - cross-site request forgery (XSRF), 439–440
 - cross-site scripting (XSS), 439–440
 - cross-tabulation, 226–227, 528–533
 - CRUD web interfaces, 419
 - cryptocurrencies, 1251–1253, 1257. *See also* Bitcoin
 - cryptographic hash functions, 1253, 1259–1263, 1265
 - CSS (cascading stylesheet) standard, 408
 - C-Store, 615
 - cube construct, 227–231, 536–538
 - current date, 154
 - cursor stability, 881
 - curve fitting, 546
 - cylinders, 565
 - Cypher query language, 509

 - DAOs (distributed autonomous organizations), 1272, 1272n7
 - Dart language, 428–429
 - data abstraction, 2, 9–12, 15
 - data access layer, 430–434
 - data analytics, 519–549. *See also* data mining
 - decision-support systems and, 519–522
 - defined, 4, 519
 - OLAP systems, 520, 527–540
 - overview, 519–521
 - predictive models in, 4–5
 - statistical analysis, 520, 527
 - warehousing and, 519–527
 - data-at-rest, 502
 - database administrators (DBAs), 24–25
 - database-as-a-service platform, 993
 - database design
 - alternatives in, 243–244, 285–291
 - applications and (*see* application design)
 - architecture of (*see* architectures)
 - authorization requirements and, 291
 - bottom-up, 273
 - buffers and, 604–610
 - client-server (*see* client-server systems)
 - complexity of, 241
 - computer-aided, 390–391
 - conceptual-design phase of, 17–18, 242
 - direct design process, 241
 - encryption and, 447–453
 - engines, 18–21
 - E-R model and (*see* entity-relationship model)
 - functional requirements of, 291
 - incompleteness in, 243–244
 - logical-design phase of, 18, 242
 - normalization in, 17
 - overview of process, 241–244
 - phases of, 17–18, 241–243
 - physical-design phase of, 18, 242–243
 - redundancy in, 243
 - relational (*see* relational database design)
 - schema evolution and, 292
 - specification of functional requirements in, 17–18
 - top-down, 273
 - user requirements in, 17–18, 241–242, 274
 - workflow and, 291–292
- database graph, 846–848**
 - database instance, 41**
 - database-management systems (DBMSs)**
 - defined, 1
 - objectives of, 1, 24
 - organizational data processing prior to, 472
 - product-specific calls needed by, 186
 - databases**
 - abstraction and, 2, 9–12, 15
 - administrators of, 24–25
 - applications for, 1–5
 - architecture (*see* architectures)
 - array, 367
 - blockchain (*see* blockchain databases)
 - buffering and, 927–929
 - centralized, 962–963
 - concurrency control and (*see* concurrency control)
 - defined, 1
 - design of (*see* database design)
 - document, 3
 - dumping and, 930–931
 - efficiency of, 1, 2, 5, 9
 - embedded, 198, 962
 - as file-processing systems, 5–8
 - force output and, 912
 - graph, 508–511
 - history of, 25–28
 - indexing and (*see* indices)
 - languages for, 13–17
 - locks and (*see* locks)

- main memory (*see*
 - main-memory databases)
- maintenance for, 25
- modification of, 108–114, 915–916
- object-based (*see* object-based databases)
- object-oriented, 9, 26, 377, 431, 1239–1240
- object-relational, 377–381
- parallel (*see* parallel databases)
- purpose of, 5–8
- query processor components of, 18, 20
- recovery of (*see* recovery systems)
- storage for (*see* storage)
- transaction manager in, 18–21
- university (*see* university databases)
- user interaction with, 4–5, 24
- databases administrator (DBA), 171**
- database schema. *See* schemas**
- database writer process, 965**
- data center fabric, 978**
- data centers, 970, 1014–1015**
- data cleansing, 523**
- data cubes, 529–530**
- data-definition language (DDL)**
 - application programs and, 17
 - authorization and, 66
 - basic types supported by, 67–68
 - in consistency constraint specification, 13–14
 - defined, 13, 65
 - dumping and, 931
 - granting and revoking privileges and, 166
 - indices and, 67
 - integrity and, 66
 - interpreter, 20
 - output of, 14
 - schema definition and, 24, 66, 68–71
 - security and, 67
 - set of relations in, 66–67
 - SQL and, 14–15, 65–71
 - storage and, 67
- data dictionary, 14, 19, 602–604**
- data distribution skew, 1008**
- Data Encryption Standard (DES), 448**
- data files, 19**
- data inconsistency, 6**
- data isolation. *See* isolation**
- data-item identifiers, 913**
- data items, 968**
- data lakes, 527, 1078**
- data-manipulation language (DML)**
 - application programs and, 17
 - compiler, 20
 - declarative, 15
 - defined, 13, 15, 66
 - procedural, 15
 - SQL and, 16
 - storage manager and, 19
- data mining, 540–549**
 - association rules and, 546–547
 - blockchain databases and, 1256, 1258, 1264–1266
 - classifiers and, 541–546
 - clustering and, 541, 548–549
 - defined, 5, 540
 - descriptive patterns and, 541
 - growth of, 27
 - models for, 540
 - overview, 521
 - prediction and, 541
 - regression and, 546
 - rules for, 540
 - task types in, 541
 - text mining, 549
- data models, 8–9. *See also* specific models**
- datanodes, 475, 1020**
- data parallelism, 1042, 1057**
- data partitioning, 989n5**
- data-server systems, 963–964, 968–970**
- DataSet type, 499**
- data storage and definition language, 13**
- data storage systems. *See* storage**
- data streams, 731**
- data striping, 571–572**
- data-transfer failures, 909**
- data-transfer rate, 566, 569**
- data types. *See* types**
- data virtualization, 1077**
- data visualization, 538–540**
- data warehousing, 519–527**
 - architecture for, 522–523
 - column-oriented storage and, 525–526
 - components of, 522–524
 - database support for, 525–526
 - data integration vs., 1077–1078
 - data lakes and, 527, 1078
 - deduplication and, 523
 - defined, 519, 522
 - ETL tasks and, 520, 524
 - fact tables and, 524
 - householding and, 523
 - merger-purge operation and, 523
 - multidimensional data and, 524
 - overview, 519–520
 - schemas used for, 523–525
 - transformation and cleansing, 523
 - updates and, 523
- datetime data type, 154, 531**
- DBAs (database administrators), 24–25**
- DBMSs. *See* database-management systems**
- DDL. *See* data-definition language**
- DDL interpreter, 20**
- deadlines, 894**
- deadlocks**
 - consistency and, 838–839
 - detection of, 849, 851–852
 - distributed databases and, 1111–1115
 - handling of, 849–853
 - prevention of, 849–851
 - recovery and, 849, 851, 853
 - rollback and, 853
 - starvation and, 853
 - victim selection and, 853

- wait-for graphs and, 851–852, 1113–1114
- debugging, 199n4**
- decentralization, 1251, 1252, 1259, 1270**
- decision support, 521, 1231–1233**
- decision-support queries, 521, 971**
- decision-support systems, 519–522**
- decision-support tasks, 521**
- decision-tree classifiers, 542**
- declarative DMLs, 15**
- declarative queries, 47, 1030–1031**
- declare statement, 201–203**
- decode, 155–156**
- decomposition**
 - algorithms for, 330–335
 - attributes and, 305–313, 330–335, 339–340
 - Boyce-Codd normal form and, 313–316, 330–333
 - dependency preservation and, 315–316, 329
 - fourth normal form and, 339–341
 - functional dependencies and, 308–313, 330–341
 - higher normal forms and, 319
 - keys and, 309–312
 - lossless, 307–308, 307n1, 312–313
 - lossy, 307
 - multivalued dependencies and, 336–341
 - normalization theory and, 308
 - notational conventions and, 309
 - relational database design and, 305–313, 330–341
 - third normal form and, 317–319, 333–335
- decomposition rule, 321**
- decompression, 613, 615n8**
- decorrelation, 777–778**
- DEC Rdb, 26**
- deduplication, 523**
- deep learning, 546**
- deep neural networks, 546**
- de facto standards, 1237**
- default values**
 - classifiers and, 545
 - privileges and, 167
 - setting reference field to, 150
 - user-defined types and, 159
- deferred integrity constraints, 151**
- deferred-modification technique, 915**
- deferred view maintenance, 779, 1215–1216**
- degree of relationship sets, 249**
- degree-two consistency, 880–881**
- delete authorization, 14**
- deletion**
 - B+-trees and, 641, 645–649
 - concurrency control and, 857–858
 - database modification and, 108–110
 - hashing and, 1190, 1194–1195, 1198
 - integrity constraints and, 150
 - LSM trees and, 1178–1179
 - of messages, 1110
 - ordered indices and, 624, 631–632
 - privileges and, 166–167
 - R-trees and, 1189
 - shipping SQL statements to database and, 187
 - SQL schema definition and, 69, 71
 - transactions and, 801, 826
 - triggers and, 208–209
 - tuples and, 108–110, 613
 - views and, 142
- deletion entries, 668, 1178–1179**
- delta relation, 211**
- demand-driven pipeline, 726–728**
- denial-of-service attacks, 502**
- denormalization, 346**
- dense indices, 626–628, 630–631**
- dependency of transactions, 819**
- dependency preservation, 315–316, 328–330**
- derived attributes, 251, 252**
- desc expression, 84**
- descriptive attributes, 248**
- descriptive patterns, 541**
- DES (Data Encryption Standard), 448**
- design. *See* database design**
- destination-driven architecture, 522**
- dicing, 530**
- dictionary attacks, 449**
- differentials, 780**
- digital certificates, 451–453**
- digital ledgers, 1251, 1252**
- digital signatures, 451, 1257, 1261**
- digital video disks (DVDs), 560–561**
- dimension attributes, 524**
- dimension tables, 524**
- direct-access storage, 561**
- directed acyclic graphs (DAGs), 499, 506–507, 1071–1072**
- directory access protocols, 1084, 1240–1243**
- directory information trees (DITs), 1242, 1243**
- directory systems, 1020, 1084–1086, 1240–1243**
- dirty blocks, 928–929**
- dirty page table, 941–947**
- dirty writes, 822**
- disable trigger, 210**
- disambiguation, 549**
- disconnected operation, 427–428**
- discretized streams, 508**
- discriminator attributes, 259**
- disjoint generalization, 279, 290**
- disjoint specialization, 272, 275**
- disjoint subtrees, 847**
- disjunctive selection, 699, 700, 762**
- disk arms, 565**
- disk-arm-scheduling, 578–579**
- disk blocks, 566–567, 577–580**
- disk buffer, 578, 910**
- disk controllers, 565**
- disk failure, 908**
- distinct types, 90, 92, 98–100, 158–160**

- distinguished name (DN),**
1241–1242
- distributed autonomous organizations (DAOs),**
1272, 1272n7
- distributed consensus problem,**
1106–1107, 1151
- distributed databases**
 - architecture of, 22, 986–989, 1098–1100
 - autonomy and, 988
 - Big Data and, 473, 480–481
 - commit protocols and, 1100–1110
 - concurrency control and, 990, 1105, 1111–1120
 - consensus in, 1106–1107, 1150–1161, 1266, 1267
 - directory systems and, 1020, 1084–1086, 1240–1243
 - failure and, 1104
 - federated, 988, 1076–1077, 1132–1133
 - file systems in, 472–475, 489, 1003, 1019–1022
 - global transactions and, 988, 1098, 1132
 - heterogeneous, 988, 1132
 - homogeneous, 988
 - leases and, 1115–1116
 - local transactions and, 988, 1098, 1132
 - locks and, 1111–1116
 - nodes and, 987
 - partitions and, 1104–1105
 - persistent messaging and, 1108–1110, 1137
 - query optimization and, 1084
 - query processing and, 1076–1086
 - recovery and, 1105
 - replication and, 987, 1121–1128
 - sharing data and, 988
 - sites and, 986
 - snapshot isolation and, 1131–1132
 - timestamps and, 1116–1118
 - transaction processing in, 989–990, 1098–1100
 - validation and, 1119–1120
- distributed file systems, 472–475,**
489, 1003, 1019–1022
- distributed hash tables, 1013**
- distributed-lock manager, 1112**
- distributed query processing,**
1076–1086
 - data integration from multiple sources, 1076–1078
 - directory systems and, 1084–1086
 - join location and join ordering in, 1081–1082
 - across multiple sources, 1080–1084
 - optimization and, 1084
 - schema and data integration in, 1078–1080
 - semijoin strategy and, 1082–1084
- DITs (directory information trees), 1242, 1243**
- Django framework, 382,**
419–421, 433–435, 1240
- DKNF (domain-key normal form), 341**
- DML. *See* data-manipulation language**
- DML compiler, 20**
- DN (distinguished name),**
1241–1242
- DNS (Domain Name Service) system, 1084, 1085**
- Docker, 995**
- document databases, 3**
- Document Object Model (DOM), 423**
- document stores, 477, 1023**
- domain constraints, 13–14, 146**
- domain-key normal form (DKNF), 341**
- Domain Name Service (DNS) system, 1084, 1085**
- domain of attributes, 39–40, 249**
- double buffering, 725**
- double-pipelined hash-join, 731**
- double-pipelined join technique,**
730–731
- double-spend transactions,**
1261–1262, 1264
- downgrade, 843**
- drill down, 531, 540**
- DriverManager class, 186**
- drop index, 165, 664**
- drop schema, 163**
- drop table, 69, 71, 190**
- drop trigger, 210**
- drop type, 159**
- dumping, 930–931**
- duplicate elimination, 719–720,**
1049
- durability**
 - defined, 800
 - one-safe, 933
 - remote backup systems and, 933–934
 - storage structure and, 804–805
 - of transactions, 20–21, 800–807
 - two-safe, 934
 - two-very-safe, 933
- DVDs (digital video disks),**
560–561
- dynamic handling of join skew,**
1048
- dynamic hashing, 661,**
1195–1203
- dynamic-programming algorithm,**
767
- dynamic repartitioning,**
1010–1013
- dynamic SQL, 66, 184, 201**
- Dynamo, 477, 489, 1024**
- Eclipse, 416**
- e-commerce, streaming data and,**
501
- edge switches, 977**
- efficiency of databases, 1, 2, 5, 9**
- e-government, 1277**
- elasticity, 992, 1010, 1024**
- election algorithms, 1147**
- elevator algorithm, 578**
- embedded databases, 198, 962**
- embedded multivalued dependencies, 341**
- embedded SQL, 66, 184,**
197–198, 965, 1269
- empty relations test, 101–102**

encryption

- Advanced Encryption Standard, 448, 449
- applications of, 447–453
- asymmetric-key, 448
- authentication and, 450–453
- blockchain databases and, 1260–1261
- challenge-response system and, 451
- database support and, 449–450
- dictionary attacks and, 449
- digital certificates and, 451–453
- digital signatures and, 451
- nonrepudiation and, 451
- prime numbers and, 449
- private-key, 1260–1261
- public-key, 448–449, 1260–1261
- Rijndael algorithm and, 448
- symmetric-key, 448
- techniques of, 447–449

end transaction operation, 799**end-user information, 443****enterprise information, database applications for, 2–4****entities, 243, 244, 247–248****entity group, 1031****entity recognition, 549****entity-relationship (E-R)****diagrams**

- aggregation and, 279
- alternative notations for modeling data, 285–291
- common mistakes in, 280–281
- complex attributes and, 265–267
- defined, 244
- entity sets and, 245–246, 265–268
- generalization and, 278–279
- participation illustrated by, 255
- reduction to relational schema, 264–271, 277–279
- relationship sets and, 247–250, 268–271

Unified Modeling Language

- and, 289–291

- for university enterprise,

- 263–264

- with weak entity set, 260

entity-relationship (E-R) model, 244–291

- aggregation and, 276–277
- alternative notations for modeling data, 285–291
- atomic domains and, 342–343
- attributes and, 245, 248–252, 274–275, 281–282, 342–343
- constraints and, 253–256, 275–276
- database design and (*see* database design)
- design issues and, 279–285
- development of, 244
- diagrams (*see* entity-relationship diagrams)
- entity sets and, 244–246, 261–264, 281–283
- extended features, 271–279
- generalization and, 273–274
- mapping cardinalities and, 252–256
- normalization and, 344–345
- overview, 8
- primary keys and, 256–260
- redundancy and, 261–264
- relationship sets and, 246–249, 282–285
- schemas and, 244, 246, 269–270, 277–279
- specialization and, 271–273
- Unified Modeling Language and, 288–291

entity sets

- alternative notations for, 285–291
- attributes and, 245, 265–267, 281–282
- defined, 245
- design issues and, 281–283
- entity-relationship diagrams and, 245–246, 265–268

entity-relationship (E-R)

- model and, 244–246, 261–264, 281–283

- extension of, 245

- hierarchies of, 273, 275

- identifying, 259

- primary keys and, 257

- properties of, 244–246

- relationship sets and,

- 246–249, 282–283

- removing redundancy in,

- 261–264

- representation of, 265–268

- strong, 259, 265–267

- subclass, 274

- superclass, 274

- Unified Modeling Language

- and, 288–291

- value and, 245

- weak, 259–260, 267–268

entries, 1241**equality-generating dependencies, 337****equi-depth histograms, 759****equi-joins, 704, 707–713, 718, 722, 730, 1043****equivalence**

- conflict, 815, 815n2

- cost analysis and, 771

- enumeration of expressions, 755–757

- join ordering and, 754–755

- relational algebra and, 58,

- 747–757

- transformation examples for,

- 752–754

- view, 818, 818n4

equivalence rules, 747–752, 754, 771**equivalent queries, 58****equi-width histograms, 759****erase block, 568****E-R diagrams. *See***

- entity-relationship diagrams

E-R model. *See* entity-relationship (E-R) model**escape, 83****Ethereum, 1258, 1262, 1265, 1267–1272, 1274****Ethernet, 978**

- ETL (extract, transform and load) tasks, 520, 524
- evaluation primitive, 691
- eventual consistency, 1016, 1139
- every function, 96
- evicted blocks, 605
- exactly-once semantics, 1074
- except all, 89, 97
- except clause, 216
- except construct, 102
- exception conditions, 202
- exceptions, 187
- except operation, 88–89
- exchange-operator model, 1055–1057
- exclusive locks
 - biased protocol and, 1124
 - concurrency control and, 835–843, 888, 892, 893
 - degree-two consistency and, 880
 - graph-based protocols and, 846–847
 - multiple granularity and, 854–855
 - multiversion, 871
 - recovery systems and, 916
 - transactions and, 825, 928
- exclusive-mode locks, 835, 842
- EXEC SQL, 197
- execute privilege, 169–170
- execution skew, 1007, 1008, 1043
- existence bitmaps, 1184
- existence dependence, 259
- exists construct, 101, 102, 108
- expiration of leases, 1115
- explain command, 746
- explicit locks, 854
- extendable hashing, 661, 1195, 1196
- Extensible Markup Language. *See* XML
- extension of entity sets, 245
- extent (blocks), 579
- external consistency, 1131
- external data, 1077
- external language routines, 203–206
- external sorting, 701
- external sort-merge algorithm, 701–704
- extract, transform and load (ETL) tasks, 520, 524
- extraneous attributes, 325
- extraneous functional dependencies, 324
- factorials, 811
- fact tables, 524
- failed transactions, 806, 907, 909
- fail-stop assumption, 908, 1267
- failure recovery, 21
- false cycles, 1114–1115
- false positives, 1083
- false values, 96
- fanout, 635
- fat-tree topology, 977
- fault tolerance
 - blockchain databases and, 1276
 - geographic distribution and, 1027
 - interconnection networks and, 978
 - MapReduce paradigm and, 1060–1061
 - in query-evaluation plans, 1059–1061
 - replicated state machines and, 1158–1161
 - shared-disk architecture and, 984
 - with streaming data, 1074–1076
 - updates and, 1138
- fault-tolerant key-value store, 1160
- fault-tolerant lock manager, 1160
- federated distributed databases, 988, 1076–1077, 1132–1133
- fetching. *See also* information retrieval
 - advanced SQL and, 187–188, 193, 195–197, 202, 205, 222
 - application design and, 421–427, 431, 437, 1218, 1229
 - by buffer manager, 19
 - data warehousing and, 526
 - large-object types and, 156, 158
 - prefetching, 969
 - storage and, 567, 572, 587
- fiat currencies, 1252, 1273
- Fiber Channel FC interface, 563
- Fiber Channel Protocol, 978
- fifth normal form, 341
- file headers, 590–591
- file manager, 19
- file organization, 588–602
 - blobs, 156, 193, 594, 652
 - blocks and, 579, 588
 - B⁺-tree, 595, 650–652, 697, 697n4
 - clobs, 156, 193, 594, 652
 - distributed, 472–475, 489, 1003, 1019–1022
 - fixed-length records and, 589–592
 - hash, 595, 659
 - heap file organization, 595–597
 - indexing and (*see* indices)
 - journaling systems, 610
 - large objects and, 594–595
 - multitable clustering, 595, 598–601
 - null values and, 593
 - partitioning and, 601–602
 - pointers and, 588, 591, 594–598, 601
 - reorganization, 598
 - sequential, 595, 597–598
 - variable-length records and, 592–594
- file-processing systems, 5–8
- file scans, 695–697, 704–707, 727
- file system consistency check, 610
- financial sector, database applications for, 3, 1279
- fine-grained locking, 947
- fine-grained parallelism, 963, 970
- firm deadlines, 894
- first committer wins, 874
- first normal form (1NF), 342–343

- first updater wins, 874–875**
- five minute rule, 1229**
- fixed-length records, 589–592**
- fixed point of recursive view**
definition, 217
- flash-as-buffer approach, 1229**
- flash memory, 567–570**
cost of, 560
erase block and, 568
hybrid, 569–570
indexing on, 656–657
LSM trees and, 1182
NAND, 567–568
NOR, 567
wear leveling and, 568
- flash translation layer, 568**
- flatMap function, 496–498**
- flexible schema, 366**
- FlinkCEP, 504**
- float, 67**
- Flutter framework, 428**
- followers, 1155**
- forced output, 607, 912**
- force policy, 927**
- for clause, 534**
- for each row clause, 207–210, 212**
- for each statement clause, 76, 209–210**
- foreign-currency exchange, 1277**
- foreign keys, 45–46, 69–70, 148–150, 267–268, 268n5**
- foreign tables, 1077**
- forking, 1257, 1258, 1263**
- formal standards, 1237**
- fourth normal form (4NF), 336, 339–341**
- fragment-and-replicate joins, 1046–1047, 1062**
- fragmentation, 579**
- free lists, 591**
- free-space maps, 596–597**
- from clause**
aggregate functions and, 91–96
basic SQL queries and, 71–79
on multiple relations, 74–79
in multiset relational algebra, 97
null values and, 90
query optimization and, 775–777
rename operation and, 79, 81–82
set operations and, 85–89
on single relation, 71–74
string operations and, 82–83
subqueries and, 104–105
- full nodes, 1256, 1268**
- full outer joins, 132–136, 722**
- functional dependencies**
algorithms for decomposition using, 330–335
attribute set closure and, 322–324
augmentation rule and, 321
axioms and, 321
Boyce–Codd normal form and, 313–316, 330–333
canonical cover and, 324–328
closure of a set, 320–324
decomposition rule and, 321
dependency preservation and, 315–316, 328–330
extraneous, 324
higher normal forms and, 319
keys and, 309–312
lossless decomposition and, 312–313
multivalued, 336–341
notational conventions and, 309
pseudotransitivity rule and, 321
reflexivity rule and, 321
in schema design, 145
theory of, 320–330
third normal form and, 317–319, 333–335
transitivity rule and, 321
trivial, 311
union rule and, 321
- functionally determined**
attributes, 322–324
- functional query language, 47**
- functions. *See also specific functions***
declaring, 199–201
external language routines and, 203–206
hash (*see* hash functions)
language constructs for, 201–203
syntax and, 199, 201–205
writing in SQL, 198–206
- fuzzy checkpoints, 922, 930, 941**
- fuzzy dump, 931**
- fuzzy lookup, 523**
- gas concept for transactions, 1270–1271**
- GAV (global-as-view) approach, 1078–1079**
- generalization**
attribute inheritance and, 274–275
bottom-up design and, 273
disjoint, 279, 290
entity-relationship (E-R) model and, 273–274
overlapping, 279, 290
partial, 275
representation of, 278–279
subclass set and, 274
superclass set and, 274
top-down design and, 273
total, 275
- Generalized Search Tree (GiST), 670**
- genesis blocks, 1254–1255**
- geographically distributed storage, 1026–1027**
- geographic data**
applications of, 391–392
examples of, 387
overlays and, 393
raster data, 392
representation of, 392–393
subtypes of, 390
topographical, 393
vector data, 392–393
- geographic information systems, 387**
- geometric data, 388–390**
- getColumnCount method, 191–192**
- getConnection method, 186, 186n1**
- getFloat method, 188**
- get function, 477–479**

- GET method, 440
 - getString method, 188
 - GFS. *See* Google File System
 - GiST (Generalized Search Tree), 670
 - Glassfish, 416
 - global-as-view (GAV) approach, 1078–1079
 - global indices, 1017–1019
 - Global Positioning System (GPS), 1130
 - global schema, 1076, 1078–1079
 - global transactions, 988, 1098, 1132
 - global wait-for graphs, 1113–1114
 - Google
 - application design and, 406–408, 410, 428–429
 - Bigtable, 477, 479–480, 668, 1024
 - PageRank from, 385–386, 493, 510
 - Pregel system developed by, 511
 - Spanner, 1160–1161
 - Google File System (GFS), 473, 474, 1020, 1022
 - Google Maps, 393
 - GPS (Global Positioning System), 1130
 - GPUs (graphics processing units), 1064
 - grant command, 170
 - granted by current role, 172–173
 - grant privileges, 166–167, 170–171
 - graph-based protocols, 846–848
 - graph databases, 508–511
 - graphics processing units (GPUs), 1064
 - GraphX, 511
 - group by clause, 92–96, 105, 142, 221, 227–230
 - group by construct, 534, 536–537
 - group by cube, 536
 - group by rollout, 537
 - group-commit technique, 925
 - grouping function, 536–537
 - grouping sets construct, 230, 538
 - growing phase, 841, 843
 - Gustafson's law, 974
 - hackers. *See* security
 - Hadoop File System (HDFS), 473–475, 489–493, 971, 1020–1022
 - Halloween problem, 785
 - handlers, 202
 - hard deadlines, 894
 - hard disk drives (HDDs). *See* magnetic disks
 - hard disks, 26
 - hard forks, 1257, 1258
 - hardware RAID, 574–576
 - hardware threads, 982
 - hardware tuning, 1227–1230
 - harmonic mean, 1231
 - hash file organization, 595, 659
 - hash functions
 - Bloom filters and, 1175–1176
 - closed, 659, 1194
 - collision resistant, 1259–1260
 - consistent, 1013
 - cryptographic, 1253, 1259–1263, 1265
 - defined, 624, 659
 - deletion, 1190, 1194–1195, 1198
 - dynamic, 661, 1195–1203
 - extendable, 661, 1195, 1196
 - insertion, 1194–1195, 1197–1202
 - irreversibility of, 1260
 - joins and, 1045
 - linear, 661, 1203
 - lookup, 1197, 1198, 1202–1203
 - open, 1194
 - partitioning and, 1045
 - passwords and, 1260n4
 - queries and, 624, 1197–1202
 - static, 661, 1190–1195, 1202–1203
 - updates and, 624, 1197–1202
 - hash indices
 - bucket overflow and, 659–660, 1194–1195
 - comparison with ordered indices, 1203
 - data structure and, 1195–1196
 - defined, 624
 - dynamic hashing and, 661, 1195–1203
 - extendable hashing and, 661
 - file organization and, 595, 659
 - insufficient buckets and, 1194
 - linear hashing and, 661, 1203
 - in main memory, 658–659
 - overflow chaining and, 659–660
 - skew and, 660, 1194
 - static hashing and, 661, 1190–1195, 1202–1203
 - tuning of, 1215
- hash join
 - basics of, 712–714
 - build input and, 713
 - cost of, 715–717
 - hybrid, 717–718
 - overflows and, 715
 - pipelining and, 728–731
 - probe input and, 713
 - query optimization and, 769, 771
 - query processing and, 712–718, 786, 1063
 - recursive partitioning and, 714–715
 - skewed partitioning and, 715
 - hash partitioning, 476, 1005–1007
 - hash-table overflow, 715
 - hash trees. *See* Merkle trees
 - having clause, 95–96, 104–105, 142
 - HBase, 477, 480, 489, 668, 971, 1024, 1028–1031
 - HDDs (hard disk drives). *See* magnetic disks
 - HDFS. *See* Hadoop File System
 - head-disk assemblies, 565
 - health care, blockchain
 - applications for, 1277–1278
 - heap files, 595–597, 1203
 - heart-beat messages, 1147
 - heterogeneous distributed databases, 988, 1132

- heuristics, 766, 771–774, 786, 1189
- Hibernate system, 382, 431–433, 1240
- hierarchical architecture, 979, 980, 986
- hierarchical clustering, 548
- hierarchical data models, 26
- hierarchies
 - cross-tabulation and, 532
 - on dimensions, 531
 - of entity sets, 273, 275
 - relational representation of, 533
 - transitive closures on, 214
- high availability, 907, 931–933, 987, 1121
- high availability proxy, 932–933
- higher-level locks, 935–936
- higher normal forms, 319
- histograms
 - attributes and, 758–760
 - distribution approximated by, 543
 - equi-depth, 759
 - equi-width, 759
 - examples of, 758–759, 1009
 - join size estimation and, 763–764
 - percentile-based, 223
 - random samples and, 761
 - range-partitioning vectors and, 1009
 - selection size estimation and, 760
- Hive, 494, 495, 500
- HOLAP (hybrid OLAP), 535
- Hollerith, Herman, 25
- homogeneous distributed databases, 988
- hopping window, 505
- horizontal partitioning, 1004, 1216–1217
- host language, 16, 197
- hot-spare configuration, 933
- hot swapping, 575
- householding, 523
- HTML. *See* HyperText Markup Language
- HTTP. *See* HyperText Transfer Protocol
- hybrid disk drives, 569–570
- hybrid hash join, 717–718
- hybrid merge-join algorithm, 712
- hybrid OLAP (HOLAP), 535
- hybrid row/column stores, 615
- hypercubes, 976–977
- Hyperledger Fabric, 1269, 1271
- hyperlinks, 385–386
- HyperText Markup Language (HTML)
 - application design and, 404, 406–408, 426
 - client-side scripting and, 421
 - Java Server Pages and, 417–418
 - security and, 439, 440
 - server-side scripting and, 416–418
 - stylesheets and, 408
 - web sessions and, 408–411
- HyperText Transfer Protocol (HTTP)
 - application design and, 405–413
 - connectionless nature of, 409–410
 - man-in-the-middle attacks and, 442
 - Representation State Transfer and, 426
 - security and, 440, 452
- hyper-threading, 982
- hypervisor, 994
- IBM DB2
 - advanced SQL and, 206
 - history of, 26
 - limit clause in, 222
 - query optimization and, 774, 783
 - Spatial Extender, 388
 - statistical analysis and, 761
 - trigger syntax and, 212
 - types and domains supported by, 160
- ICOs (initial coin offerings), 1272
- IDEFIX standard, 288
- IDE (integrated development environment), 416
- idempotent operations, 937
- identifiers
 - block, 474–475, 1020
 - data-item, 913
 - indices and, 700
 - log records and, 913
 - query processing and, 700
 - selection and, 700
 - transaction, 913
- identifying entity sets, 259
- identifying relationship, 259–260
- identity declaration, 1226
- identity specification, 161
- identity theft, 447
- IDF (inverse document frequency), 384
- IEEE (Institute of Electrical and Electronics Engineers), 1237
- if clauses, 212
- if-then-else statements, 202
- immediate-modification technique, 915
- immediate view maintenance, 779, 1215–1216
- imperative query language, 47
- implicit locks, 854–855
- incompleteness in database design, 243–244
- inconsistent data, 6
- inconsistent state, 802, 803, 812
- in construct, 99
- incremental view maintenance, 779–782
- increment lock, 892–893
- increment operation, 892
- independent parallelism, 1054–1055
- indexed nested-loop join, 707–708, 728
- index entries, 626
- indexing strings, 653
- index-locking protocol, 860
- index-locking technique, 860
- index records, 626
- index scans, 696, 698–699, 769, 769n2

index-sequential files, 625, 634–635

indices, 623–676, 1175–1203

access time and, 624, 627–628
 access types and, 624
 basic concepts related to, 623–624
 bitmap (*see* bitmap indices)
 Bloom filters and, 667, 1175–1176, 1181
 B⁺-tree (*see* B⁺-trees)
 buffer trees and, 668–670
 bulk loading of, 653–655
 clustering, 625, 632–633, 695, 697–698
 comparisons and, 698–699
 composite, 700
 concurrency control and, 884–887
 covering, 663
 creation of, 664–665, 884–885
 defined, 19
 definition in SQL, 164–165, 664–665
 deletion time and, 624, 631–632, 641, 645–649
 dense, 626–628, 630–631
 on flash storage, 656–657
 Generalized Search Tree, 670
 global, 1017–1019
 hash (*see* hash indices)
 identifiers and, 700
 insertion time and, 624, 630–631, 641–645, 647, 649
 inverted, 721
 key-value stores and, 1028
 linear search and, 695
 local, 1017
 LSM trees and, 666–668, 1176–1182, 1215
 in main memory, 657–658
 materialized views and, 783
 multilevel, 628–630
 multiple-key access and, 633–634, 661–663
 nonclustering, 625, 695
 ordered (*see* ordered indices)

parallel, 1017–1019
 performance tuning and, 1215
 pointers and, 700
 primary, 625, 695, 1017–1018
 query processing and, 695–697
 record relocation and, 652–653
 search keys and, 624–634
 secondary, 625, 632–633, 652–653, 695–698, 1017–1019
 selection operation and, 695–697, 783
 sequential, 625, 634–635
 sorting and, 701–704
 space overhead and, 624, 627–628, 634, 1202
 sparse, 626–632
 spatial data and, 672–675, 1186–1190
 SQL DDL and, 67
 stepped-merge, 667, 1179–1181
 of temporal data, 675–676
 updates and, 630–632
 write-optimized structures, 665–670

in-doubt transactions, 1105

infeasibility, 1259–1260

Infiniband standard, 978

information extraction, 549

information retrieval. *See also*

queries

defined, 382
 keywords and, 383
 measuring effectiveness of, 386
 PageRank and, 385–386
 precision and, 386
 recall and, 386
 relevance ranking and, 383–386
 stop words and, 385
 structured data queries and, 386–387
 TF-IDF approach and, 384–385

infrastructure-as-a-service model, 991

Ingres system, 26

inheritance

attribute, 274–275
 multiple, 275
 single, 275
 tables and, 379–380
 types and, 378–379

initial coin offerings (ICOs), 1272

initialization vector, 449

initially deferred integrity constraints, 151

inner joins, 132–136, 771

inner relation, 704

insert authorization, 14

insertion

algorithm for, 1180–1181
 B⁺-trees and, 641–645, 647, 649
 concurrency control and, 857, 858
 database modification and, 110–111
 default values and, 156
 hashing and, 1194–1195, 1197–1202
 LSM trees and, 1177–1178, 1180–1181
 ordered indices and, 624, 630–631
 prepared statements and, 188–190
 privileges and, 166–167
 R-trees and, 1188–1189
 shipping SQL statements to database and, 187
 SQL schema definition and, 69
 transactions and, 801, 826
 views and, 141–143

instances, 12, 309, 547. *See also* training instances

instead of feature, 143

Institute of Electrical and Electronics Engineers (IEEE), 1237

insurance claims, 1278

integrated development environment (IDE), 416
integrity constraints

- add, 146
- alter table and, 146
- assertions and, 152–153
- assigning names to, 151
- authorization, 14
- check clause and, 147–149, 152–153
- create table and, 146–149
- deferred, 151
- domain, 13–14
- examples of, 145
- in file-processing systems, 6
- foreign keys and, 148–150
- functional dependencies (*see* functional dependencies)
- not null, 146, 150
- primary keys and, 147, 148
- referential, 14, 46, 149–153, 207–208, 800
- schema diagrams of, 46–47
- on single relation, 146
- spatial, 391
- SQL and, 14–15, 66, 145–153
- unique, 147
- user-defined types and, 159–160
- violation during transactions, 151–152
- integrity manager, 19**
- Intel, 569, 1064**
- intention-exclusive (IX) mode, 855**
- intention lock modes, 855**
- intention-shard (IS) mode, 855**
- interconnection networks, 975–979**
- interesting sort order, 770**
- interference, 974, 1232**
- intermediate keys, 1050**
- intermediate SQL, 125–173**
 - authorization and, 165–173
 - create table extensions and, 162
 - data/time types in, 154
 - default values and, 156
 - generating unique key values and, 160–161
 - index definition in, 164–165
 - integrity constraints and, 145–153
 - join expressions and, 125–136
 - large-object types and, 156, 158
 - roles and, 167–169
 - schemas, catalogs, and environments, 162–163
 - transactions and, 143–145
 - type conversion and formatting functions, 155–156
 - user-defined types and, 158–160
 - views and, 137–143
- internal nodes, 635**
- International Organization for Standardization (ISO), 65, 1237, 1241**
- Internet. *See* World Wide Web**
- Internet of Things (IoT), 470, 1278**
- interoperation parallelism, 1040, 1052–1055**
- interquery parallelism, 1039**
- intersect all, 88, 97**
- intersection of bitmaps, 671, 1183**
- intersection operation, 750, 782**
- intersect operation, 54–55, 87–88**
- interval data type, 154**
- intra-node partitioning, 1004**
- intraoperation parallelism**
 - aggregation and, 1049
 - defined, 1040
 - duplicate elimination and, 1049
 - map and reduce operations and, 1050–1052
 - parallel external sort-merge and, 1042–1043
 - parallel join and, 1043–1048
 - parallel sort and, 1041–1043
 - projection and, 1049
 - range-partitioning sort and, 1041–1042
 - selection and, 1049
- intraquery parallelism, 1039**
- invalidation reports, 1141**
- invalidation timestamps, 873n3**
- inverse document frequency (IDF), 384**
- inverted indices, 721**
- I/O operations per second (IOPS), 567, 577, 578, 693n3**
- I/O parallelism**
 - hashing and, 1005–1007
 - partitioning techniques and, 1004–1007
 - range scheme and, 1005, 1007
 - round-robin scheme and, 1005, 1006
- IoT (Internet of Things), 470, 1278**
- irrefutability, 1257, 1259**
- irreversibility, 1260**
- IS (intention-shard) mode, 855**
- is not null, 89, 90**
- is not unknown, 90**
- is null, 89**
- ISO (International Organization for Standardization), 65, 1237, 1241**
- isolation**
 - atomicity and, 819–821
 - cascadeless schedules and, 820–821
 - concurrency control and, 803–804, 807–812, 823
 - of data, 6
 - defined, 800
 - factorials and, 811
 - improved throughput and, 808
 - inconsistent state and, 803
 - levels of, 821–826
 - locking and, 823–825
 - multiple versions and, 825–826
 - read committed, 1225
 - recoverable schedules and, 819–820
 - resource allocation and, 808
 - serializability and, 821–826
 - snapshot (*see* snapshot isolation)
 - timestamps and, 825

- of transactions, 800–804, 807–812, 819–826
 - utilization and, 808
 - wait time and, 808–809
 - is unknown, 90**
 - iteration, 201–202, 214–216**
 - iterators, 727**
 - IX (intention-exclusive) mode, 855**
 - Jakarta Project, 416**
 - Java. *See also* JDBC (Java Database Connectivity)**
 - advanced SQL and, 183, 197, 199, 205
 - application design and, 16, 417
 - metadata and, 191–193
 - object-oriented programming and, 377
 - object-relational mapping system for, 382
 - Resilient Distributed Dataset and, 496–498
 - Unified Modeling Language and, 289
 - Java Database Connectivity. *See* JDBC**
 - Java 2 Enterprise Edition (J2EE), 416**
 - JavaScript**
 - application design and, 404–405, 421–426
 - input validation and, 422–423
 - interfacing with web services, 423–426
 - Representation State Transfer and, 426, 427
 - responsive user interfaces and, 423
 - security and, 439
 - JavaScript Object Notation (JSON)**
 - applications for use, 368–369
 - defined, 368
 - emergence of, 27
 - encoding results with, 426
 - example of, 368, 369
 - flexibility of, 367–368
 - key-value stores and, 1024
 - mapping to data models, 480
 - as semi-structured data model, 8, 27
 - SQL in support of, 369–370
 - for transferring data, 423
 - Java Server Pages (JSP)**
 - application design and, 405, 417–418
 - security and, 440
 - server-side scripting and, 417–418
 - servlets and, 417–418
 - Java Servlets, 411–416, 419, 424**
 - JBoss, 416**
 - JDBC (Java Database Connectivity), 184–193**
 - blob column and, 193
 - caching and, 435–436
 - callable statements and, 190–191
 - clob column and, 193
 - connecting to database, 185–186
 - corresponding interface defined by, 17
 - exception and resource management, 187
 - metadata features and, 191–193
 - prepared statements and, 188–190
 - protocol information, 186
 - retrieving query results, 187–188
 - shipping SQL statements to, 186–187
 - updatable result sets and, 193
 - web sessions and, 409
 - join conditions, 130–131**
 - join dependencies, 341**
 - join operation, 52–53**
 - joins**
 - anti-join operation, 108, 776
 - anti-semijoin operation, 776–777
 - broadcast, 1046
 - complex, 718
 - cost analysis and, 710–712, 767–770
 - equi-joins, 704, 707–713, 718, 722, 730, 1043
 - fragment-and-replicate, 1046–1047, 1062
 - full outer, 132–136, 722
 - hash (*see* hash join)
 - hybrid merge, 712
 - inner, 132–136, 771
 - inner relation of, 704
 - left outer, 131–136, 722
 - merge-join, 708–712, 1045
 - minimization and, 784
 - natural (*see* natural joins)
 - nested loop (*see* nested-loop join)
 - ordering, 754–755
 - outer, 57, 131–136, 722–723, 765, 782
 - outer relation of, 704
 - parallel, 1043–1048
 - partitioned, 714–715, 1043–1046
 - query processing and, 704–719, 722–723, 1081–1082
 - right outer, 132–136, 722
 - semijoin operation, 108, 776, 1082–1084
 - size estimation and, 762–764
 - skew in, 1047–1048
 - sort-merge-join, 708–712
 - spatial, 394
 - spatial data and, 719
 - streaming data and, 506
 - theta, 748–749
 - types and conditions, 130–131, 136
 - view maintenance and, 780
- join skew avoidance, 1048**
 - join using operation, 129–130**
 - journaling file systems, 610**
 - JSON. *See* JavaScript Object Notation**
 - JSP. *See* Java Server Pages**
 - J2EE (Java 2 Enterprise Edition), 416**
 - jukebox systems, 561**
 - Kafka system, 506, 507, 1072–1073, 1075, 1137**

- k-d B trees**, 674
- KDD (knowledge discovery in databases)**, 540
- k-d trees**, 673–674
- kernel functions**, 545
- keys**
 - candidate, 44
 - cluster key, 600–601
 - constraints and, 258
 - encryption and, 448–449, 451–453
 - equality on, 697
 - foreign, 45–46, 69–70, 148–150, 267–268, 268n5
 - functional dependencies and, 309–312
 - intermediate, 1050
 - multiple access, 633–634, 661–663
 - partitioning, 475–476
 - primary (*see* primary keys)
 - reduce, 485
 - in relational model, 43–46
 - search (*see* search keys)
 - smart cards and, 451
 - superkeys, 43–44, 257–258, 309–310, 312
 - unique values, 160–161
- key-value locking**, 887
- key-value maps**, 366–367
- key-value storage systems**
 - Big Data and, 471, 473, 476–480
 - clusters and, 477
 - document, 477, 1023
 - fault tolerant, 1160
 - parallel, 1003, 1023–1031
 - replication and, 476
 - social-networking sites and, 471
 - wide-column, 1023
- keyword queries**, 383, 385–387, 721
- killing the mutant**, 1234
- knowledge discovery in databases (KDD)**, 540
- knowledge graphs**, 374–375, 386–387, 549
- knowledge representation**, 368
- Kubernetes**, 995
- lambda architecture**, 504, 1071
- language constructs**, 201–203
- Language Integrated Query (LINQ)**, 198
- LANs**. *See* local-area networks
- large object storage**, 594–595
- large-object types**, 156, 158
- LastLSN**, 943, 946
- latches**, 886, 928
- latch-free data structure**, 888–890
- latency**, 989
- latent failure**, 575
- lateral clause**, 105
- LAV (local-as-view) approach**, 1079
- lazy generation of tuples**, 727
- lazy propagation of updates**, 1122, 1136
- LDAP Data Interchange Format (LDIF)**, 1242
- LDAP (Lightweight Directory Access Protocol)**, 442, 1085, 1240–1243
- leaders**, 1155
- leaf nodes**, 635–656, 665–669, 673, 674
- learners**, 1148, 1153
- leases**, 1115–1116, 1147
- least recently used (LRU) strategy**, 605, 607
- ledgers, digital**, 1251, 1252
- left-deep join orders**, 773
- left outer join**, 131–136, 722
- legacy systems**, 1035–1036
- legal instance**, 309
- LevelDB**, 668
- Lightning network**, 1275
- light nodes**, 1256, 1268
- Lightweight Directory Access Protocol (LDAP)**, 442, 1085, 1240–1243
- like operator**, 82–83
- limit clause**, 222
- linear hashing**, 661, 1203
- linearizability**, 1121
- linear probing**, 1194
- linear regression**, 546
- linear scaleup**, 972–973
- linear search**, 695
- linear speedup**, 972
- line segments**, 388–390
- linestrings**, 388, 390
- Linked Data project**, 376, 1080
- LINQ (Language Integrated Query)**, 198
- load balancers**, 934–935, 1214
- load barrier**, 983
- local-area networks (LANs)**, 977, 978, 985, 989
- local-as-view (LAV) approach**, 1079
- local autonomy**, 988
- local indices**, 1017
- local schema**, 1076
- localtimestamp**, 154
- local transactions**, 988, 1098, 1132
- local wait-for graphs**, 1113
- lock conversions**, 843
- lock-free data structure**, 890
- locking protocols**
 - B-link tree, 886
 - concurrency control and, 835–848
 - defined, 839
 - distributed lock-manager, 1112
 - graph-based, 846–848
 - implementation of, 844–846
 - index, 860
 - key-value, 887
 - multiple granularity, 856–857
 - next-key, 887
 - single lock-manager, 1111
 - transactions and, 823–825
 - two-phase, 841–844, 871–872, 1129–1131
- lock managers**, 844–845, 965, 1160
- lock point**, 841
- locks**
 - adaptive granularity and, 969–970
 - caching and, 969
 - call back and, 969
 - compatibility function and, 836

- deadlocks (*see* deadlocks)
- de-escalation and, 970
- distributed databases and, 1111–1116
- escalation and, 857, 1227
- exclusive (*see* exclusive locks)
- explicit, 854
- false cycles and, 1114–1115
- fine-grained, 947
- granting of, 836, 840–841
- growing phase and, 841, 843
- higher-level, 935–936
- implicit, 854–855
- increment, 892–893
- intention modes and, 855
- leases, 1115–1116, 1147
- logical undo operations and, 935–941
- lower-level, 935–936
- multiple granularity and, 853–857
- multiversion schemes and, 871–872, 1129–1131
- predicate, 828, 861, 861n1
- recovery systems and, 935–941
- request operation and, 835–841, 844–846, 849–853, 886
- shared, 825, 835, 854, 880, 1124
- shrinking phase and, 841, 843
- starvation and, 853
- timeouts and, 850–851
- timestamps and, 861–866
- transaction servers and, 965–968
- true matrix value and, 836
- wait-for graph and, 851–852, 1113–1114
- lock table, 844, 845, 967**
- log disks, 610**
- log force, 927**
- logical clock, 1118**
- logical counter, 862**
- logical-design phase, 18, 242**
- logical error, 907**
- logical level of abstraction, 9–12**
- logical logging, 936**
- logically implied schema, 320**
- logical operations**
 - concurrency control and, 940–941
 - consistency and, 936–937
 - defined, 935
 - early lock release and, 935–941
 - idempotent, 937
 - log records and, 936–940
 - rollback and, 937–939
- logical routing of tuples, 506–507, 1071–1073**
- logical schema, 12–13, 1223–1224**
- logical undo operations, 935–941**
- log of transactions, 805**
- log processing, 486–488**
- log records**
 - ARIES and, 941–946
 - buffering and, 926–927
 - checkpoint, 943
 - compensation, 922, 942, 945
 - database modification and, 915–916
 - force/no-force policy and, 927
 - identifiers and, 913
 - logical undo operations and, 936–940
 - old/new values and, 913
 - physical, 936
 - recovery systems and, 913–919, 926–927
 - redo operation and, 915–919
 - steal/no-steal policy and, 927
 - undo operation and, 915–919
 - write-ahead logging rule and, 926–929
- log sequence number (LSN), 941–946**
- log-structured merge (LSM) trees, 1176–1182**
 - basic, 1179
 - Bloom filters and, 1181
 - deletion and, 1178–1179
 - for flash storage, 1182
 - insertion into, 1177–1178, 1180–1181
 - levels of, 1176
 - lookup and, 1181
 - parallel key-value stores and, 1028
 - performance tuning and, 1215
 - rolling merges and, 1178
 - stepped-merge indices and, 1179–1181
 - updates and, 1178–1179
 - write-optimized structure of, 666–668
- log writer process, 965**
- long-duration transactions, 890–891**
- lookup**
 - in blockchain databases, 1268
 - Bloom filters and, 667, 1181
 - concurrency control and, 884–887
 - data-storage systems and, 480
 - fuzzy, 523
 - hashing and, 1197, 1198, 1202–1203
 - indices and, 630, 637, 640–641, 645–651, 656–661, 666–669, 676
 - LSM trees and, 1181
 - query optimization and, 769
 - query processing and, 698, 707
- lossless decomposition, 307–308, 307n1, 312–313**
- lossy decompositions, 307**
- lost update problem, 1016**
- lost updates, 874**
- lower-level locks, 935–936**
- loyalty programs, 1278**
- LRU (least recently used) strategy, 605, 607**
- LSM trees. *See* log-structured merge trees**
- LSN (log sequence number), 941–946**
- machine-learning algorithms, 495**
- magnetic disks, 563–567**
 - access time and, 561, 566, 567
 - blocks and, 566–567
 - capacity of, 560
 - checksums and, 565
 - crashes and, 565
 - data-transfer rate and, 566

- disk controller and, 565
- failure classification and, 908
- hybrid, 569–570
- mean time to failure and, 567
- performance measures of, 565–567
- physical characteristics of, 563–565
- read-write heads and, 564–565
- recording density and, 565
- sectors and, 564–566
- seek time and, 566, 566n2, 567
- sizes of, 563
- main memory, 559–560, 657–658**
- main-memory databases**
 - accessing data in, 910
 - concurrency control in, 887–890
 - recovery in, 947–948
 - storage in, 588, 615–617
- majority protocol, 1123–1126**
- management of data. *See* database-management systems (DBMSs)**
- man-in-the-middle attacks, 442**
- manufacturing, database applications for, 3, 5**
- many-to-many mapping, 253–255**
- many-to-one mapping, 252–255**
- map function, 483–494, 510. *See also* MapReduce paradigm**
- mapping cardinalities, 252–256**
- MapReduce paradigm, 483–494**
 - development of, 27, 481
 - fault tolerance and, 1060–1061
 - in Hadoop, 489–493
 - intraoperation parallelism and, 1050–1052
 - log processing and, 486–488
 - parallel processing of tasks, 488–489
 - SQL on, 493–494
 - word count program and, 483–486, 484n2, 490–492
- markup languages. *See specific languages***
- massively parallel systems, 970**
- master nodes, 1012, 1136**
- master replica, 1016**
- master sites, 1026**
- master-slave replication, 1137**
- master table, 1222**
- match clause, 509**
- materialization, 724–725**
- materialized edges, 728**
- materialized views**
 - aggregation and, 781–782
 - defined, 140, 778
 - index selection and, 783
 - join operation and, 780
 - parallel maintenance of, 1069–1070
 - performance tuning and, 1215–1216
 - projection and, 780–781
 - query optimization and, 778–783
 - selection and, 780–781
 - view maintenance and, 140, 779–782
- max function, 91–92, 105, 723, 766, 782**
- maximum margin line, 544**
- mean time between failures (MTBF), 567n3**
- mean time to data loss, 571**
- mean time to failure (MTTF), 567, 567n3**
- mean time to repair, 571**
- measure attributes, 524**
- mediators, 1077**
- memcached system, 436–437, 482**
- memoization, 771**
- memory. *See also* storage**
 - bulk loading of indices and, 653–655
 - cache, 559 (*see also* caching)
 - data access and, 910–912
 - flash, 560, 567–570, 656–657
 - force output and, 912
 - magnetic-disk, 560, 563–567
 - main, 559–560, 657–658
 - non-volatile random-access, 579–580
 - optical, 560–561
 - overflows and, 715
 - query costs and, 697
 - query processing in, 731–734
 - recovery systems and, 910–912
 - storage class, 569, 588, 948
- memory barrier, 983–984**
- merge-join, 708–712, 1045**
- merge-purge operation, 523**
- merging**
 - duplicate elimination and, 719–720
 - exchange-operator model and, 1055–1057
 - ordered, 1056
 - parallel external sort-merge, 1042–1043
 - performance tuning and, 1222–1223
 - query processing and, 701–704, 708–712
 - random, 1056
 - rolling merge, 1178
- Merkle-Patricia trees, 1269, 1275**
- Merkle trees, 1143–1146, 1268, 1269**
- mesh system, 976**
- MESI protocol, 984**
- message-based consensus, 1266**
- message delivery process, 1109–1110**
- metadata, 14, 191–193, 470, 602–604, 1020–1022**
- microservices architecture, 994**
- Microsoft**
 - application design and, 417, 442
 - Database Tuning Assistant, 1217
 - query languages developed by, 538
 - query optimization and, 783
 - StreamInsight, 504
- Microsoft SQL Server**
 - advanced SQL and, 206
 - implements, 160

- limit clause in, 222
- performance monitoring tools, 1212
- performance tuning tools, 1218
- procedural languages supported by, 199
- snapshot isolation and, 1225
- spatial data and, 388, 390
- string operations and, 82
- min function**, 91–92, 723, 766, 782
- minibatch transactions**, 1227
- minimal equivalence rules**, 754
- mining pools**, 1266. *See also* data mining
- minus**, 88n7
- mirroring**, 571–573, 576, 577
- mobile application platforms**, 428–429
- mobile phone applications**, 469
- models for data mining**, 540
- model-view-controller (MVC) architecture**, 429–430
- MOLAP (multidimensional OLAP)**, 535
- MonetDB**, 367, 615
- MongoDB**, 477–479, 482, 489, 668, 1024, 1028
- monotonic queries**, 218
- Moore's law**, 980n4
- most recently used (MRU) strategy**, 608–609
- MRU (most recently used) strategy**, 608–609
- MTBF (mean time between failures)**, 567n3
- MTTF (mean time to failure)**, 567, 567n3
- multidimensional data**, 524, 527–532
- multidimensional OLAP (MOLAP)**, 535
- multilevel indices**, 628–630
- multimaster replication**, 1137
- multiple consensus protocol**, 1151
- multiple granularity**
 - concurrency control and, 853–857
 - hierarchy of, 854
 - intention-exclusive mode and, 855
 - intention-shared mode and, 855
 - locking protocol and, 856–857
 - request operation and, 854, 855
 - shared and intention-exclusive mode and, 855
 - tree architecture and, 853–857
- multiple inheritance**, 275
- multiple-key access**, 633–634, 661–663
- multiprogramming**, 809
- multiquery optimization**, 785–786
- multiset except**, 88n7
- multiset relational algebra**, 80, 97, 108, 136, 747
- multiset types**, 366
- multisig instruction**, 1269–1270
- multitable clustering file organization**, 595, 598–601
- multitasking**, 961, 963
- multiuser systems**, 962
- multivalued attributes**, 251, 252, 342
- multivalued data types**, 366–367
- multivalued dependencies**, 336–341
- multiversion concurrency control (MVCC)**, 869–872
- multiversion timestamp-ordering scheme**, 870–871, 1118
- multiversion two-phase locking (MV2PL) protocol**, 871–872, 1129–1131
- mutations**, 1234
- mutual exclusion**, 965, 967
- MVCC (multiversion concurrency control)**, 869–872
- MVC (model-view-controller) architecture**, 429–430
- MV2PL (multiversion two-phase locking) protocol**, 871–872, 1129–1131
- MySQL**
 - attributes and, 149n8
 - growth of, 27
 - joins and, 133n4
 - LSM trees and, 668
 - performance monitoring tools, 1212
 - string operations and, 82
 - unique key values in, 161
- naïve Bayesian classifiers**, 543
- naïve users**, 24
- namenode**, 475, 1020
- NAND flash memory**, 567–568
- NAS (network attached storage)**, 563, 934
- natural join operation**, 57
- natural joins**, 126–136
 - on condition and, 130–131
 - conditions and, 130–131
 - full outer, 132–136, 722
 - inner, 132–136, 771
 - left outer, 131–136, 722
 - outer, 57, 131–136
 - right outer, 132–136, 722
- navigation systems, database applications for**, 3
- nearest-neighbor queries**, 394, 672, 674
- nearness queries**, 394
- negation**, 762
- Neo4j graph database**, 509, 510
- nested data types**, 367–368
- nested-loop join**
 - block, 705–707
 - indexed, 707–708, 728
 - parallel, 1045–1046
 - query optimization and, 768, 769, 771, 773
 - query processing and, 704–708, 713–719, 722, 786
- nested subqueries**, 98–107
 - from clause and, 104–105
 - with clause and, 105–106
 - duplicate tuples and, 103
 - empty relations test and, 101–102
 - optimization of, 774–778, 1220

- scalar, 106–107
- set operations and, 98–101
- nesting, 854, 946**
- NetBeans, 416**
- network attached storage (NAS), 563, 934**
- network latency, 969**
- network partition, 481, 989, 989n5, 1104–1105**
- network round-trip time, 969**
- networks**
 - deep neural, 546
 - interconnection, 975–979
 - local area, 977, 978, 985, 989
 - spatial, 390
 - wide-area, 989
- neural-net classifiers, 545–546**
- new value, 913**
- next-key locking protocols, 887**
- next method, 188**
- nextval for, 1226**
- NFNf (non first-normal-form), 367**
- nodes**
 - in blockchain databases, 1255–1257, 1263–1268
 - coalescing, 641, 886
 - datanodes, 475, 1020
 - defined, 468
 - distributed databases and, 987
 - failure of, 1103–1104
 - full, 1256, 1268
 - in Hadoop File System, 474, 475
 - internal, 635
 - leaf, 635–656, 665–669, 673, 674
 - leases and, 1115–1116
 - light, 1256, 1268
 - master, 1012, 1136
 - mesh architecture and, 976
 - multiple granularity and, 853–857
 - namenode, 475, 1020
 - nonleaf, 635–636, 642, 645–656, 663
 - operation, 506–507
 - in parallel databases, 970
 - primary, 1123
 - ring architecture and, 976
 - splitting of, 641–645, 886
 - straggler, 1060–1061
 - updates and, 641–647
 - virtual, 1009–1010
- no-force policy, 927**
- nonbinary relationship sets, 283–285**
- non-blocking two-phase commit, 1161**
- nonces, 1265, 1271**
- nonclustering indices, 625, 695**
- nondeclarative actions, 183**
- non first-normal-form (NFNF), 367**
- nonleaf nodes, 635–636, 642, 645–656, 663**
- nonprocedural DMLs, 15**
- nonprocedural languages, 15, 16, 18, 26**
- nonrepudiation, 451**
- Non-Uniform Memory Access (NUMA), 981, 1063**
- nonunique search keys, 632, 637, 640, 649–650**
- Non-Volatile Memory Express (NVMe) interface, 562**
- non-volatile random-access memory (NVRAM), 579–580, 948**
- non-volatile storage, 560, 562, 587–588, 804, 908–910, 930–931**
- non-volatile write buffers, 579–580**
- NOR flash memory, 567**
- normal forms**
 - atomic domains and, 342–343
 - Boyce–Codd, 313–316, 330–333, 336
 - domain-key, 341
 - fifth, 341
 - first, 342–343
 - fourth, 336, 339–341
 - higher, 319
 - join dependencies and, 341
 - project-join, 341
 - second, 316n8, 341–342, 356
 - third, 317–319, 333–335
- normalization**
 - in conceptual-design process, 17
 - denormalization, 346
 - entity-relationship (E-R) model and, 344–345
 - performance and, 346
 - relational database design and, 308
- NoSQL systems, 28, 473, 477, 1269, 1276**
- no-steal policy, 927**
- not connective, 74**
- not exists construct, 101–102, 108, 218**
- notifications, 436**
- not in construct, 99, 100**
- not null, 69, 89–90, 142, 146, 150, 159**
- not operation, 89–90**
- not unique construct, 103**
- null bitmap, 593**
- null rejecting property, 751**
- null values**
 - aggregation with, 96
 - attributes and, 251–252
 - defined, 40, 67
 - file organization and, 593
 - integrity constraints and, 145–147, 150
 - SQL and, 89–90
 - temporal data and, 347
 - triggers and, 209
 - user-defined types and, 159
- NUMA (Non-Uniform Memory Access), 981, 1063**
- numeric, 67, 70**
- nvarchar, 68**
- NVMe (Non-Volatile Memory Express) interface, 562**
- NVRAM (non-volatile random-access memory), 579–580, 948**
- N-way merge, 702**
- OAuth protocol, 443**
- obfuscation, 1235**
- object-based databases**
 - array types and, 378
 - complex data types and, 376–382

- inheritance and, 378–380
- mapping and, 377, 381–382
- overview, 9
- reference types and, 380–381
- object classes, 1242**
- Object Database Management Group (ODMG), 1239–1240**
- Object Management Group (OMG), 288**
- object of triples, 372**
- object-oriented databases (OODB), 9, 26, 377, 431, 1239–1240**
- object-relational databases, 377–381**
 - defined, 377
 - reference types and, 380–381
 - table inheritance and, 379–380
 - type inheritance and, 378–379
 - user-defined types, 378
- object-relational data models, 27, 376–382**
- object-relational mapping (ORM), 377, 381–382, 431–434, 1239–1240**
- observable external writes, 807**
- ODBC (Open Database Connectivity)**
 - advanced SQL and, 194–197
 - API defined by, 194–195
 - application interfaces defined by, 17
 - caching and, 436
 - conformance levels and, 196–197
 - standards for, 1238–1239
 - type definition and, 196
 - web sessions and, 409
- ODMG (Object Database Management Group), 1239–1240**
- off-chain transactions, 1275**
- offline storage, 561**
- OGC (Open Geospatial Consortium), 388**
- OLAP. *See* online analytical processing**
- old value, 913**
- OLE-DB, 1239**
- OLTP (online transaction processing), 4, 521, 1231–1232**
- OMG (Object Management Group), 288**
- on condition, 130–131**
- on delete cascade, 150, 210, 268n5**
- 1NF (first normal form), 342–343**
- one-to-many mapping, 252–255**
- one-to-one mapping, 252–254**
- online analytical processing (OLAP), 527–540**
 - aggregation on multidimensional data, 527–532
 - cross-tabulation and, 528–533
 - data cubes and, 529–530
 - defined, 520, 530
 - dicing and, 530
 - drill down and, 531, 540
 - hybrid, 535
 - implementation of, 535
 - multidimensional, 535
 - performance benchmarks and, 1231–1232
 - relational, 535
 - reporting and visualization tools, 538–540
 - rollup and, 530–531, 536–538
 - slicing and, 530
 - in SQL, 533–534, 536–538
- online index creation, 884–885**
- online storage, 561**
- online transaction processing (OLTP), 4, 521, 1231–1232**
- on update cascade, 150**
- OODB. *See* object-oriented databases**
- open addressing, 1194**
- Open Database Connectivity. *See* ODBC**
- Open Geospatial Consortium (OGC), 388**
- open hashing, 1194**
- OpenID protocol, 443**
- open polygons, 388n3**
- open time intervals, 675**
- operation consistent state, 936–937**
- operation nodes, 506–507**
- operation serializability, 885**
- operator trees, 724, 1040**
- optical storage, 560–561**
- optimistic concurrency control, 869**
- optimistic concurrency control without read validation, 883, 891**
- optimization cost budget, 774**
- Oracle**
 - advanced SQL and, 206
 - application server, 416
 - database design and, 443n6, 444–445
 - decode function in, 155
 - Event Processing, 504
 - GeoRaster extension, 367
 - history of, 26
 - JDBC interface and, 185, 186
 - keywords in, 81n3, 88n7
 - limit clause in, 222
 - nested subqueries and, 104
 - performance monitoring tools, 1212
 - performance tuning tools, 1218
 - procedural languages supported by, 199
 - query-evaluation plans and, 746
 - query optimization and, 773, 774, 783
 - reference types and, 380
 - set and array types supported by, 367
 - snapshot isolation and, 1225
 - Spatial and Graph, 388
 - statistical analysis and, 761
 - syntax supported by, 204, 212, 218, 218n9
 - transactions and, 822, 826, 872–873, 879

- types and domains supported
 - by, 160
 - Virtual Private Database, 173, 444–445
- oracles, 1271–1272**
- ORC, 490, 499, 613–614**
- or connective, 74**
- order by clause, 83–84, 219–222, 534**
- ordered indices, 624–634**
 - comparison with hash indices, 1203
 - defined, 624
 - dense, 626–628, 630–631
 - multilevel, 628–630
 - secondary, 625, 632–633
 - sequential, 625, 634–635
 - sparse, 626–632
 - techniques for, 624
 - updates and, 630–632
- ordered merge, 1056**
- ORM. *See* object-relational mapping**
- or operation, 89–90**
- orphaned blocks, 1257, 1263**
- outer-join operation, 57, 131–136, 722–723, 765, 782**
- outer relation, 704**
- outer union operations, 229n16**
- outsourcing, 28**
- overflow avoidance, 715**
- overflow blocks, 598**
- overflow buckets, 659–660, 1194–1195**
- overflow chaining, 659–660**
- overflow resolution, 715**
- overlapping generalization, 279, 290**
- overlapping specialization, 272, 275**
- overlays, 393**
- page (blocks), 567**
- PageLSN, 942–945**
- PageRank, 385–386**
- page shipping, 968**
- parallel databases**
 - architecture of, 22, 970–986
 - Big Data and, 473, 480–481
 - coarse-grain, 963, 970
 - concurrency control and, 990
 - defined, 480
 - exchange-operator model and, 1055–1057
 - fine-grain, 963, 970
 - hierarchical, 979, 980, 986
 - indices in, 1017–1019
 - interconnection networks and, 975–979
 - interference and, 974
 - interoperation parallelism and, 1040, 1052–1055
 - interquery parallelism and, 1039
 - intraoperation parallelism and, 1040–1052
 - intraquery parallelism and, 1039
 - I/O parallelism and, 1004–1007
 - key-value stores and, 1023–1031
 - massively parallel, 970
 - motivation for, 970–971
 - operator trees and, 1040
 - partitioning techniques and, 1004–1007
 - performance measures for, 971–974
 - pipelines and, 1053–1054
 - query optimization and, 1064–1070
 - replication and, 1013–1016
 - response time and, 971–972
 - scaleup and, 972–974
 - shared disk, 979, 980, 984–985
 - shared memory, 979–984, 1061–1064
 - shared nothing, 979, 980, 985–986, 1040–1041, 1061–1063
 - skew and, 974, 1007–1013, 1043, 1062
 - speedup and, 972–974
 - start-up costs and, 974, 1066
 - throughput and, 971
 - transaction processing in, 989–990
- parallel external sort-merge, 1042–1043**
- parallel indices, 1017–1019**
- parallelism**
 - coarse-grained, 963
 - data, 1042, 1057
 - fine-grained, 963
 - improvement of performance via, 571–572
 - independent, 1054–1055
 - interoperation, 1040, 1052–1055
 - interquery, 1039
 - intraoperation, 1040–1052
 - intraquery, 1039
 - I/O (*see* I/O parallelism)
 - Single Instruction Multiple Data, 1064
- parallel joins, 1043–1048**
 - fragment-and-replicate, 1046–1047, 1062
 - hash, 1045
 - nested-loop, 1045–1046
 - partitioned, 1043–1046
 - skew in, 1047–1048
- parallel key-value stores, 1023–1031**
 - atomic commit and, 1029
 - concurrency control and, 1028–1029
 - data representation and, 1024–1025
 - defined, 1003
 - elasticity and, 1024
 - failures and, 1029–1030
 - geographically distributed, 1026–1027
 - index structure and, 1028
 - managing without declarative queries, 1030–1031
 - overview, 1023–1024
 - performance optimizations and, 1031
 - storing and retrieving data, 1025–1028
 - support for transactions, 1028–1030
- parallel processing, 437, 488–489**
- parallel query plans**

- choosing, 1066–1068
- colocation of data and, 1068–1069
- cost of, 1065–1066
- evaluation of, 1052–1061
- materialized views and, 1069–1070
- space for, 1064–1065
- parallel sort, 1041–1043**
- parameterized views, 200**
- parameter style general, 205**
- parametric query optimization, 786**
- parity bits, 572, 574, 577**
- Parquet, 490, 499**
- parsing**
 - application design and, 418
 - bulk loads and, 1221–1223
 - query processing and, 689–690
- partial aggregation, 1049**
- partial dependency, 356**
- partial failure, 909**
- partial generalization, 275**
- partially committed transactions, 806**
- partial participation, 255**
- partial rollback, 853**
- partial schedules, 819**
- partial specialization, 275**
- participation in relationship sets, 247**
- partitioning attribute, 479**
- partitioning keys, 475–476**
- partitioning vector, 1005**
- partitions**
 - balanced range, 1008–1009
 - data, 989n5
 - defined, 1100
 - distributed databases and, 1104–1105
 - distributed file systems and, 473
 - dynamic repartitioning, 1010–1013
 - exchange-operator model and, 1055–1057
 - file organization and, 601–602
 - hash, 476, 1005–1007, 1045
 - horizontal, 1004, 1216–1217
 - intra-node, 1004
 - joins and, 714–715, 1043–1046
 - network, 481, 989, 989n5, 1104–1105
 - parallel databases and, 1004–1007
 - point queries and, 1006
 - query optimization and, 1065
 - range, 476, 1005, 1007, 1178
 - recursive, 714–715
 - of relation schema, 1216–1217
 - round-robin, 1005, 1006
 - scanning a relation and, 1006
 - sharding and, 473, 475–476, 1275
 - skew and, 715, 1007–1013
 - topic-partition, 1073
 - vertical, 1004
 - virtual node, 1009–1010
- partition tables, 1011–1014**
- passwords**
 - application design and, 403, 411, 414, 432, 450–451
 - dictionary attacks and, 449
 - distributed databases and, 1240
 - hash functions and, 1260n4
 - leakage of, 440–441
 - man-in-the-middle attacks and, 442
 - one-time, 441
 - single sign-on system and, 442–443
 - SQL and, 163, 186, 196
 - storage and, 602, 603
 - unencrypted, 414n4
- path expressions, 372, 381**
- pattern matching, 504**
- Paxos protocol, 1152–1155, 1160–1161, 1267**
- PCIe interface, 562**
- pen drives, 560**
- performance**
 - access time and, 561, 566–567, 578, 624, 627–628, 692
 - application design and, 434–437
 - benchmarks (*see* performance benchmarks)
 - of blockchain databases, 1274–1276
 - B+-trees and, 634, 665–666
 - caching and, 435–437
 - data-transfer rate and, 566
 - denormalization for, 346
 - improvement via parallelism, 571–572
 - magnetic disk storage and, 565–567
 - mean time to failure and, 567, 567n3
 - monitoring tools, 1212
 - parallel databases and, 971–974
 - parallel key-value stores and, 1031
 - parallel processing and, 437
 - response time (*see* response time)
 - seek time and, 566, 566n2, 567, 692, 710
 - sequential indices and, 634–635
 - testing, 1235
 - throughput (*see* throughput)
 - tuning (*see* performance tuning)
 - web applications and, 405–411
- performance benchmarks, 1230–1234**
 - database-application classes and, 1231–1232
 - defined, 1230
 - suites of tasks and, 1231
 - of Transaction Processing Performance Council, 1232–1234
- performance tuning, 1210–1230**
 - automated, 1217–1218
 - bottleneck locations and, 1211–1213, 1215, 1227
 - bulk loads and, 1221–1223
 - of concurrent transactions, 1224–1227
 - of hardware, 1227–1230

- horizontal partitioning of
 - relation schema, 1216–1217
- indices and, 1215
- levels of, 1213–1214
- materialized views and, 1215–1216
- motivation for, 1210–1211
- parameter adjustment and, 1210, 1213–1215, 1220, 1228, 1230
- physical design and, 1217–1218
- of queries, 1219–1223
- RAID and, 1214, 1229–1230
- of schema, 1214–1218, 1223–1224
- set orientation and, 1220–1221
- simulation and, 1230
- tools for, 1218
- updates and, 1221–1223, 1225–1227
- period declaration, 157**
- Perl, 206**
- permissioned blockchains, 1253–1254, 1256–1257, 1263, 1266, 1274**
- persistent messaging, 990, 1016, 1108–1110, 1137**
- Persistent Storage Module (PSM), 201**
- phantom phenomenon, 827, 858–861, 877–879, 877n5, 885, 887**
- PHP, 405, 417, 418**
- physical blocks, 910**
- physical data independence, 9–10, 13**
- physical-design phase, 18, 242–243**
- physical equivalence rules, 771**
- physical level of abstraction, 9, 11, 12, 15**
- physical logging, 936**
- physical schema, 12, 13**
- physical storage systems, 559–580**
 - cache memory, 559
 - cost per byte, 560, 561, 566n2, 569, 576
 - disk-block access and, 577–580
 - flash memory, 560, 567–570, 656–657
 - hierarchy of, 561, 562
 - indices and, 630n2
 - interfaces for, 562–563
 - magnetic disks, 560, 563–567
 - main memory, 559–560
 - optical storage, 560–561
 - RAID, 562, 570–577
 - solid-state drives, 18, 560
 - tape storage, 561
 - volatility of, 560, 562
- physiological redo operations, 941**
- Pig Latin, 494**
- pin count, 605**
- pinned blocks, 605**
- pin operations, 605**
- pipelined edges, 728**
- pipeline stage, 728**
- pipelining, 724–731**
 - benefits of, 726
 - for continuous-stream data, 731
 - demand-driven, 726–728
 - evaluation algorithms for, 728–731
 - implementation of, 726–728
 - parallelism and, 1053–1054
 - producer-driven, 726–728
 - uses for, 691–692, 724, 725
- pivot attribute, 227**
- pivot clause, 227, 534**
- pivoting, 226–227, 530**
- pivot-table, 226–227, 528–529**
- PJNF (project-join normal form), 341**
- plan caching, 774**
- platform-as-a-service model, 992–993**
- platters, 563, 565**
- PL/SQL, 199, 204**
- pointers. *See also* indices**
 - blockchain databases and, 1254–1255, 1261, 1269
 - B⁺-tree (*see* B⁺-trees)
- concurrency control and, 886, 888, 889
- query processing and, 697, 698, 700, 708
- recovery systems and, 914, 945
- redistribution of, 646
- SQL basics and, 193, 205
- storage and, 588, 591, 594–598, 601
- point queries, 1006, 1190**
- polygons, 388–390, 388n3, 393**
- polylines, 388–389, 388n3**
- population, 547**
- PostgreSQL**
 - advanced SQL and, 206
 - array types on, 378
 - concurrency control and, 873, 879
 - Generalized Search Tree and, 670
 - growth of, 27
 - heap file organization and, 596
 - JDBC interface and, 185
 - JSON and, 370
 - performance monitoring tools, 1212
 - PostGIS extension, 367, 388, 390
 - procedural languages supported by, 199
 - query-evaluation plans and, 746
 - query processing and, 692, 694, 698–699
 - set and array types supported by, 367
 - snapshot isolation and, 1225
 - statistical analysis and, 761
 - transaction management in, 822, 826
 - types and domains supported by, 160
 - unique key values in, 161
- P + Q redundancy schema, 573, 574**
- Practical Byzantine Fault Tolerance, 1267**
- precedence graph, 816–817**

- precision, 386
- precision locking, 861n1
- predicate locking, 828, 861, 861n1
- predicate of triples, 372
- predicate reads, 858–861
- prediction, 541–543, 545–546
- predictive models, 4–5
- preemption, 850
- prefetching, 969
- prefix compression, 653
- Pregel system, 511
- prepared statements, 188–190
- presentation layer, 429
- price per TPS, 1232
- primary copy, 1123
- primary indices, 625, 695, 1017–1018
- primary keys
 - attributes and, 310n4
 - defined, 44
 - entity-relationship (E-R) model and, 256–260
 - functional dependencies and, 313
 - integrity constraints and, 147, 148
 - in relational model, 44–46
 - SQL schema definition and, 68–70
- primary nodes, 1123
- primary site, 931
- primary storage, 561
- prime attributes, 356
- privacy, 438, 446, 1252
- private-key encryption, 1260–1261
- privileges
 - all, 166
 - defined, 165
 - execute, 169–170
 - granting, 166–167, 170–171
 - public, 167
 - references, 170
 - revoking, 166–167, 171–173
 - select, 171, 172
 - transfer of, 170–171
 - update, 170
- probe input, 713
- procedural DMLs, 15
- procedural languages, 47n3, 184, 199, 204
- procedures
 - declaring, 199–201
 - external language routines and, 203–206
 - language constructs for, 201–203
 - syntax and, 199, 201–205
 - writing in SQL, 198–206
- process monitor process, 965
- producer-driven pipeline, 726–728
- programming languages. *See also specific*
 - accessing SQL from, 183–198
 - mismatch and, 184
 - object-oriented, 377
 - variable operation of, 184
- Progressive Web Apps (PWA), 429
- projection
 - intraoperation parallelism and, 1049
 - query optimization and, 764
 - query processing and, 720
 - view maintenance and, 780–781
- project-join normal form (PJNF), 341
- project operation, 49–50
- proof-of-stake consensus, 1256, 1266
- proof-of-work consensus, 1256, 1264–1266
- proposers, 1148, 1152
- proximity of terms, 385
- PR quadrees, 1187
- pseudotransitivity rule, 321
- PSM (Persistent Storage Module), 201
- public blockchains, 1253, 1255, 1257–1259, 1263, 1264
- public-key encryption, 448–449, 1260–1261
- publish-subscribe (pub-sub) systems, 507, 1072, 1137–1139
- pulling data, 727
- punctuations, 503–504
- pushing data, 727
- put function, 477, 478
- puzzle friendliness, 1265
- PWA (Progressive Web Apps), 429
- Python
 - advanced SQL and, 183, 193–194, 206
 - application design and, 16, 405, 416, 419
 - object-oriented programming and, 377
 - object-relational mapping system for, 382
 - web services and, 424
- quadratic split heuristic, 1189
- quads, 376
- quadrees, 392, 674, 1186–1187
- queries. *See also information retrieval*
 - ADO.NET and, 184
 - basic structure of SQL queries, 71–79
 - on B⁺-trees, 637–641, 690
 - caching and, 435–437
 - Cartesian product and, 76–79, 81, 230
 - compilation of, 733
 - continuous, 503, 731
 - correlated subqueries and, 101
 - cost of (*see* query cost)
 - decision-support, 521, 971
 - declarative, 1030–1031
 - defined, 15
 - deletion and, 108–110
 - equivalent, 58
 - evaluation of (*see* query-evaluation plans)
 - hash functions and, 624, 1197–1202
 - indices and, 623, 695–697, 707–708
 - insertion and, 110–111
 - intermediate SQL and (*see* intermediate SQL)
 - JDBC and, 184–193
 - join expressions and, 125–136

- keyword, 383, 385–387, 721
- languages (*see* query languages)
- metadata and, 191–193
- monotonic, 218
- multiple-key access and, 661–663
- on multiple relations, 74–79
- nearest-neighbor, 394, 672, 674
- nested subqueries, 98–107
- null values and, 89–90
- ODBC and, 194–197
- optimization of (*see* query optimization)
- PageRank and, 385–386
- performance tuning of, 1219–1223
- point, 1006, 1190
- processing (*see* query processing)
- programming language access and, 183–198
- Python and, 193–194
- range, 638, 672, 674, 1006, 1190
- read only, 1039
- recursive, 213–218
- region, 393–394
- rename operation and, 79, 81–82
- ResultSet object and, 185, 187–188, 191–193, 638–639
- retrieving results, 187–188
- scalar subqueries, 106–107
- security and, 437–446
- servlets and, 411–421
- set operations and, 85–89, 98–101
- on single relation, 71–74
- spatial, 393–394
- spatial graph, 394
- streaming data and, 502–506, 1070–1071
- string operations and, 82–83
- transaction servers and, 965
- universal Turing machines and, 16
- views and, 137–143
- query cost**
 - optimization and, 745–746, 757–766
 - processing and, 692–695, 697, 702–704, 710–712, 715–717
- query-evaluation engine, 20**
- query-evaluation plans**
 - choice of, 766–778
 - cost of, 1065–1066
 - defined, 691
 - expressions and, 724–731
 - fault tolerance in, 1059–1061
 - materialization and, 724–725
 - optimization and (*see* query optimization)
 - parallel (*see* parallel query plans)
 - performance tuning of, 1219–1220
 - pipelining and, 691–692, 724–731
 - relational algebra and, 690–691
 - resource consumption and, 694–695, 1065–1066
 - response time and, 694–695
 - role in query processing, 689, 690
 - viewing, 746
- query-execution engine, 691**
- query-execution plans, 691**
- query languages. *See also specific languages***
 - accessing from programming languages, 183–198
 - categorization of, 47
 - Cypher, 509
 - defined, 15, 47
 - in relational model, 47–48
 - SPARQL, 375–376
 - stream, 503–506
- query optimization, 743–787**
 - adaptive, 786–787
 - aggregation and, 764
 - Cartesian product and, 748, 749, 755, 763–764, 775
 - cost analysis and, 745–746, 757–766
 - defined, 20, 691, 743
 - distributed, 1084
 - equivalence and, 747–757
 - estimating statistics of
 - expression results, 757–766
 - heuristics in, 766, 771–774, 786
 - hybrid hash join and, 717–718
 - indexed nested-loop join and, 707–708
 - join minimization and, 784
 - materialized views and, 778–783
 - multiquery, 785–786
 - nested subqueries and, 774–778, 1220
 - parallel databases and, 1064–1070
 - parametric, 786
 - plan choice for, 766–778
 - projection and, 764
 - relational algebra and, 743–749, 752, 755
 - role in query processing, 689, 690
 - set operations and, 764–765
 - shared scans and, 785–786
 - top-*K*, 784
 - transformations and, 747–757
 - updates and, 784–785
- query processing, 689–734**
 - adaptive, 786–787
 - aggregation and, 723
 - basic steps of, 689, 690
 - Big Data and, 470–472
 - blockchain databases and, 1254, 1275–1276
 - comparisons and, 698–699
 - cost analysis of, 692–695, 697, 702–704, 710–712, 715–717
 - CPU speeds and, 692
 - DDL interpreter in, 20
 - defined, 689
 - distributed databases and, 1076–1086
 - DML compiler in, 20
 - duplicate elimination and, 719–720
 - evaluation of expressions, 724–731

- file scans and, 695–697, 704–707, 727
- hashing and, 712–718, 1063
- history of, 26–27
- identifiers and, 700
- indices and, 695–697
- join operation and, 704–719, 722–723, 1081–1082
- materialization and, 724–725
- in memory, 731–734
- operation evaluation and, 690–691
- parsing and translation in, 689–690
- pipelining and, 691–692, 724–731
- PostgreSQL and, 692, 694, 698–699
- projection and, 720
- recursive partitioning and, 714–715
- relational algebra and, 689–691
- scalability of, 471–472
- selection operation and, 695–700
- set operations and, 720–722
- on shared-memory architecture, 1061–1064
- sorting and, 701–704
- SQL and, 689–690, 701, 720
- syntax and, 689
- query processor, 18, 20**
- query-server systems, 963**
- queueing systems, 1212–1213**
- queueing theory, 1213**
- quorum consensus protocol, 1124–1125**
- Raft protocol, 1148, 1155–1158, 1267**
- RAID. *See* redundant arrays of independent disks**
- random access, 567, 578**
- randomized retry, 1148**
- random merge, 1056**
- random samples, 761**
- range partitioning, 476, 1005, 1007, 1178**
- range-partitioning sort, 1041–1042**
- range-partitioning vector, 1008–1009**
- range queries, 638, 672, 674, 1006, 1190**
- ranking, 219–223, 383–386**
- raster data, 392**
- RDDs (Resilient Distributed Datasets), 496–499, 1061**
- RDF. *See* Resource Description Framework**
- RDMA (remote direct memory access), 979**
- RDNs (relative distinguished names), 1241–1242**
- reactionary standards, 1237**
- React Native framework, 428**
- read-ahead, 578**
- read authorization, 14**
- read committed, 821, 880, 1225**
- read cost, 1127–1128**
- read one, write all copies protocol, 1125**
- read one, write all protocol, 1125**
- read only queries, 1039**
- read-only transactions, 871**
- read phase, 866**
- read quorum, 1124**
- read uncommitted, 821**
- read-write contention, 1224–1225**
- ready state, 989, 1102**
- real, double precision, 67**
- real-time transaction systems, 894**
- rebuild performance, 576**
- recall, 386**
- RecLSN, 942–945**
- reconciliation of updates, 1142–1143**
- reconfiguration, 1128**
- record-based models, 8**
- record relocation, 652–653**
- recovery manager, 21**
- recovery systems, 907–948**
 - actions following crashes, 923–925
 - algorithms for, 922–925, 944–946, 1276
 - ARIES, 941–947, 1276
 - atomicity and, 803, 912–922
- buffer management and, 926–930
- checkpoints and, 920–922, 930
- commit protocols and, 1105
- concurrency control and, 916
- data access and, 910–912
- database modification and, 915–916
- distributed databases and, 1105
- early lock release and, 935–941
- fail-stop assumption and, 908, 1267
- failure and, 907–909, 930–932
- force/no-force policy and, 927
- logical undo operations and, 935–941
- log records and, 913–919, 926–927
- log sequence number and, 941–946
- main-memory databases and, 947–948
- redo operation and, 915–919, 922–925
- remote backup, 909, 931–935
- rollback and, 916–919, 922
- shadow-copy scheme and, 914
- snapshot isolation and, 916
- steal/no-steal policy and, 927
- storage and, 908–912, 920–922, 930–931
- successful completion and, 909
- transactions and, 21, 803, 805
- triggers and, 212–213
- undo operation and, 915–919, 922–925
- write-ahead logging rule and, 926–929
- recovery time, 933**
- recursive partitioning, 714–715**
- recursive queries, 213–218**
 - iteration and, 214–216
 - SQL and, 216–218
 - transitive closure and, 214–216

- recursive relationship sets, 247–248
- Redis, 436–437, 482, 1024
- redistribution of pointers, 646
- redo-only log records, 918
- redo operation, 915–919, 922–925, 941
- redo pass, 944–945
- redo phase, 923, 924
- reduceByKey function, 498
- reduce function, 483–494, 510.
 - See also* MapReduce paradigm
- reduce key, 485
- redundancy
 - in database design, 243
 - entity-relationship (E-R) model and, 261–264
 - in file-processing systems, 6
 - reliability improvement via, 570–571
 - of schemas, 269–270
- redundant arrays of independent disks (RAID), 570–577
 - bit-level striping and, 571–572
 - block-level striping and, 572
 - hardware issues, 574–576
 - hot swapping and, 575
 - levels, 572–574, 572n4, 573n6, 576–577
 - mirroring and, 571–573, 576, 577
 - parity bits and, 572, 574, 577
 - performance improvement via parallelism, 571–572
 - performance tuning and, 1214, 1229–1230
 - purpose of, 562
 - rebuild performance of, 576
 - recovery systems and, 909
 - reliability improvement via redundancy, 570–571
 - scrubbing and, 575
 - software RAID, 574, 575
 - striping data and, 571–572
- re-engineering, 1236
- referenced relation, 45–46
- references, 149–150
- references privilege, 170
- reference types, 380–381
- referencing new row as clause, 207, 208
- referencing new table as clause, 210
- referencing old row as clause, 208
- referencing old table as clause, 210
- referencing relation, 45–46
- referential integrity, 14, 46, 149–153, 207–208, 800
- referrer, 440
- referrals, 1243
- ref from clause, 380
- reflexivity rule, 321
- region quadrees, 1187
- region queries, 393–394
- regression, 546, 1234
- reification, 376
- reintegration, 1128
- relation, defined, 39
- relational algebra, 48–58
 - antijoin operation, 108
 - assignment operation, 55–56
 - Big Data and, 494–500
 - Cartesian-product operation, 50–52
 - equivalence and, 58, 747–757
 - expression transformation and, 747–757
 - join operation, 52–53
 - motivation for, 495–496
 - multiset, 80, 97, 108, 136, 747
 - predefined functions supported by, 48n4
 - project operation, 49–50
 - query optimization and, 743–749, 752, 755
 - query processing and, 689–691
 - rename operation, 56–57
 - select operation, 49
 - semijoin operation, 108, 1082–1084
 - set operations, 53–55
 - in Spark, 495–500, 508
 - SQL operations and, 80
 - on streams, 504, 506–508
 - unary vs. binary operations in, 48
- relational-algebra expressions, 50
- relational database design, 303–351
 - atomic domains and, 342–343
 - Boyce–Codd normal form and, 313–316
 - closure of a set and, 312, 320–324
 - decomposition and, 305–313, 330–341
 - design process, 343–347
 - features of good designs, 303–308
 - first normal form and, 342–343
 - fourth normal form and, 336, 339–341
 - functional dependencies and, 308–313, 320–330
 - larger schemas and, 330, 346
 - multivalued dependencies and, 336–341
 - naming of attributes and relationships in, 345–346
 - normalization and, 308
 - second normal form and, 316n8, 341–342, 356
 - smaller schemas and, 305, 308, 344
 - temporal data modeling and, 347–351
 - third normal form and, 317–319
- relational model, 37–59
 - conceptual-design process for, 17
 - disadvantages of, 26
 - history of, 26
 - keys for, 43–46
 - object-relational, 27, 376–382
 - operations in, 48–58
 - overview, 8, 37
 - query languages in, 47–48
 - schema diagrams for, 46–47
 - schema in (*see* relational schema)
 - SQL language in, 13
 - structure of, 37–40
 - tables in, 9, 10, 37–40
 - tuples in, 39, 41, 43–46

relational OLAP (ROLAP), 535**relational schema**

- Boyce-Codd normal form and, 313–316, 330–333
- canonical cover and, 324–328
- database design process and, 343–347
- decomposition and, 305–313
- defined, 41
- in first normal form, 342–343
- fourth normal form and, 339–341
- functional dependencies and, 308–313, 320–330
- horizontal partitioning of, 1216–1217
- logically implied, 320
- multivalued dependencies and, 336–341
- reduction of
 - entity-relationship diagrams to, 264–271, 277–279
- redundancy in, 243
- temporal data and, 347–351
- third normal form and, 317–319, 333–335
- for university databases, 41–43, 303–305

relation instance, 39, 41**relation scans, 769****relationship instance, 246–247****relationships, defined, 246****relationship sets**

- alternative notations for, 285–291
- atomic domains and, 342–343
- binary, 249, 283–285
- combination of schemas and, 270–271
- degree of, 249
- descriptive attributes and, 248
- design issues and, 282–285
- entity-relationship diagrams and, 247–250, 268–271
- entity-relationship (E-R) model and, 246–249, 282–285
- mapping cardinalities and, 252–256

- naming of, 345–346
- nonbinary, 283–285
- participation in, 247
- primary keys and, 257–259
- recursive, 247–248
- redundancy and, 269–270
- representation of, 268–269
- ternary, 249, 250, 284
- Unified Modeling Language and, 288–291

relations (tables), 8**relative distinguished names (RDNs), 1241–1242****relevance**

- hyperlinks and, 385–386
- PageRank and, 385–386
- TF-IDF approach and, 384–385

relevance ranking, 383–386**reliability, improvement via redundancy, 570–571****remapping bad sectors, 565****remote backup systems, 909, 931–935****remote direct memory access (RDMA), 979****rename operation, 56–57, 79, 81–82****renewing leases, 1115****reorganization of files, 598****repeatable read, 821****repeating history, 924****repeat loop, 214–216, 323****repeat statements, 201****replication**

- asynchronous, 1122, 1135–1138
- biased protocol and, 1124
- Big Data and, 481–482
- caching, 1014n4
- chain replication protocol, 1127–1128
- concurrency control and, 1123–1125
- consistency and, 1015–1016, 1121–1123, 1133–1146
- data centers and, 1014–1015
- distributed databases and, 987, 1121–1128
- failure and, 1125–1128

- key-value storage systems and, 476

- location of, 1014–1015

- majority protocol and, 1123–1126

- master replica, 1016

- master-slave, 1137

- multimaster, 1137

- parallel databases and, 1013–1016

- primary copy, 1123

- quorum consensus protocol and, 1124–1125

- reconfiguration and reintegration, 1128

- sharding and, 476

- state machines and, 1158–1161

- synchronous, 522–523, 1136

- two-phase commit protocol and, 1016

- update-anywhere, 1137

- updates and, 1015–1016

- view maintenance and, 1138–1140

report generators, 538–540**Representation State Transfer (REST), 426–427****request forgery, 439–440****request operation**

- deadlock handling and, 849–853

- locks and, 835–841, 844–846, 849–853, 886

- multiple granularity and, 854, 855

- multiversion schemes and, 871

- snapshot isolation and, 874
- timestamps and, 861

Resilient Distributed Datasets (RDDs), 496–499, 1061**resolution of conflicting updates, 1142–1143****resource consumption, 694–695, 1065–1066****Resource Description Framework (RDF)**

- defined, 372

- graph representation of, 374–375
- n*-ary relationships and, 376
- overview, 368
- reification in, 376
- SPARQL and, 375–376
- triple representation and, 372–374
- resources, in RDF, 372**
- response time**
 - application design and, 434–435, 1229, 1232
 - blockchain databases and, 1274
 - parallel databases and, 971–972
 - partitioning and, 1007
 - query-evaluation plans and, 694–695
 - query processing and, 694–695
 - skew and, 1066
 - storage and, 572
 - transactions and, 808–809
- response time cost model, 1066**
- responsive user interfaces, 423**
- REST (Representation State Transfer), 426–427**
- restriction, 172, 328, 339–340**
- ResultSet object, 185, 187–188, 191–193, 638–639**
- resynchronization, 575**
- reverse engineering, 1236**
- revoke privileges, 166–167, 171–173**
- right outer join, 132–136, 722**
- rigorous two-phase locking protocol, 842, 843**
- Rijndael algorithm, 448**
- ring system, 976**
- robustness, 1121, 1125–1126**
- ROLAP (relational OLAP), 535**
- roles**
 - authorization and, 167–169
 - entity, 247–248
 - indicators associated with, 268
 - Unified Modeling Language and, 289
- rollback**
 - ARIES and, 945–946
 - cascading, 820–821, 841–842
 - concurrency control and, 841–844, 849–850, 853, 868–871
 - logical operations and, 937–939
 - partial, 853
 - recovery systems and, 916–919, 922
 - remote backup systems and, 933
 - timestamps and, 862–865
 - total, 853
 - transactions and, 143–145, 193, 196, 805–806, 922, 937–939, 945–946
 - undo operation and, 916–919, 937–939
- rollback work, 143–145**
- rolling merge, 1178**
- rollup clause, 228**
- rollup construct, 227–231, 530–531, 536–538**
- rotational latency time, 566**
- round-robin scheme, 1005, 1006**
- routers, 1012–1013**
- row-level authorization, 173**
- row-oriented storage, 611, 615**
- row stores, 612, 615**
- R-timestamp, 862, 865, 870**
- R-trees, 663, 670, 674–676, 1187–1190**
- Ruby on Rails, 417, 419**
- rules for data mining, 540**
- runs, 701–702**
- runstats, 761**
- SAML (Security Assertion Markup Language), 442–443**
- SAN. *See* storage area network**
- sandbox, 205**
- SAP HANA, 615, 1132**
- SAS (Serial Attached SCSI) interface, 562**
- SATA (Serial ATA) interface, 562, 568, 569**
- savepoints, 947**
- scalability, 471–472, 477, 482, 1276**
- scalar subqueries, 106–107**
- scaleup, 972–974**
- scanning a relation, 1006**
- scheduling**
 - disk-arm, 578–579
 - query optimization and, 1065
 - transactions and, 810–811, 811n1
- schema diagrams, 46–47**
- schemas**
 - alternative notations for modeling data, 285–291
 - authorization on, 170
 - basic SQL query structures and, 71–79
 - combination of, 270–271
 - composite attributes in, 250
 - concurrency control and (*see* concurrency control)
 - creation of, 24
 - data mining, 540–549
 - data warehousing, 523–525
 - defined, 12
 - entity-relationship diagrams and, 264–271, 277–279
 - entity-relationship (E-R) model and, 244, 246, 269–270, 277–279
 - evolution of, 292
 - flexibility of, 366
 - global, 1076, 1078–1079
 - integration of, 1076, 1078–1080
 - intermediate SQL and, 162–163
 - larger, 330, 346
 - local, 1076
 - locks and (*see* locks)
 - logical, 12–13, 1223–1224
 - performance tuning of, 1214–1218, 1223–1224
 - physical, 12, 13
 - physical-organization modification of, 25
 - P + Q redundancy, 573, 574
 - recovery systems and (*see* recovery systems)
 - redundancy of, 269–270
 - relational (*see* relational schema)

- relationship sets and, 268–269
- shadow-copy, 914
- smaller, 305, 308, 344
- snowflake, 524
- SQL DDL and, 24, 66, 68–71
- star, 524–525
- strong entity sets and, 265–267
- subschemas, 12
- timestamps and, 861–866
- for university databases, 1287–1288
- version-numbering, 1141
- version-vector, 1141–1142
- weak entity sets and, 267–268
- SciDB, 367**
- SCM (storage class memory), 569, 588, 948**
- SCOPE, 494**
- scope clause, 380**
- scripting languages, 404–405, 416–418, 421, 439**
- scrubbing, 575**
- SCSI (small-computer-system interconnect), 562, 563**
- search keys**
 - B⁺-trees and, 634–650
 - hashing and, 1190–1195, 1197–1199
 - index creation and, 664
 - nonunique, 632, 637, 640, 649–650
 - ordered indices and, 624–634
 - storage and, 595, 597–598
 - uniquifiers and, 649–650
- search operation, 1188**
- secondary indices, 625, 632–633, 652–653, 695–698, 1017–1019**
- secondary site, 931**
- secondary storage, 561**
- second normal form (2NF), 316n8, 341–342, 356**
- sectors, 564–566**
- security**
 - abstraction levels and, 12
 - application design and, 437–446
 - audit trails and, 445–446
 - authentication (*see* authentication)
 - authorization (*see* authorization)
 - of blockchain databases, 1253–1255, 1259
 - concurrency control and (*see* concurrency control)
 - cross-site scripting and, 439–440
 - dictionary attacks and, 449
 - encryption and, 447–453
 - end-user information and, 443
 - in file-processing systems, 7
 - GET method and, 440
 - integrity manager and, 19
 - locks and (*see* locks)
 - man-in-the-middle attacks and, 442
 - passwords (*see* passwords)
 - privacy and, 438, 446
 - request forgery and, 439–440
 - single sign-on system and, 442–443
 - SQL DDL and, 67
 - SQL injection and, 438–439
 - unique identification and, 446, 446n8
- Security Assertion Markup Language (SAML), 442–443**
- seek time, 566, 566n2, 567, 692, 710**
- select all, 73**
- select authorization, privileges and, 166–167**
- select clause**
 - aggregate functions and, 91–96
 - attribute specification in, 83
 - basic SQL queries and, 71–79
 - on multiple relations, 74–79
 - in multiset relational algebra, 97
 - null values and, 90
 - OLAP and, 534, 536–537
 - ranking and, 220–222
 - rename operation and, 79, 81–82
 - set membership and, 98–99
 - set operations and, 85–89
 - on single relation, 71–74
 - string operations and, 82–83
- select distinct, 72–73, 90, 99–100, 142**
- select-from-where**
 - deletion and, 108–110
 - function/procedure writing and, 199–206
 - insertion and, 110–111
 - join expressions and, 125–136
 - natural joins and, 126–130
 - nested subqueries and, 98–107
 - updates and, 111–114
 - views and, 137–143
- selection**
 - comparisons and, 698–699
 - complex, 699–700
 - conjunctive, 699–700, 747, 762
 - disjunctive, 699, 700, 762
 - equivalence and, 747–757
 - file scans and, 695–697, 704–707, 727
 - identifiers and, 700
 - indices and, 695–697, 783
 - intraoperation parallelism and, 1049
 - linear search and, 695
 - size estimation and, 760, 762
 - view maintenance and, 780–781
- selectivity, 762**
- select operation, 49**
- select privilege, 171, 172**
- semijoin operation, 108, 776, 1082–1084**
- semi-structured data models, 365–376**
 - flexible schema and, 366
 - history of, 8, 27
 - JSON, 8, 27, 367–370
 - knowledge representation and, 368
 - motivations for use of, 365–366
 - multivalued, 366–367
 - nested, 367–368

- overview, 366–368
- RDF and knowledge graphs, 368, 372–376
- XML, 8, 27, 367–368, 370–372
- sensor data, 470, 501**
- sentiment analysis, 549**
- Sequel, 65**
- sequence counters, 1226**
- sequential-access storage, 561, 567, 578**
- sequential computation, 974**
- sequential file organization, 595, 597–598**
- Serial ATA (SATA) interface, 562, 568, 569**
- Serial Attached SCSI (SAS) interface, 562**
- serializability**
 - blind writes and, 868
 - concurrency control and, 836, 840–843, 846–848, 856, 861–871, 875–887
 - conflict, 813–816
 - isolation and, 821–826
 - operation, 885
 - order of, 817
 - performance tuning and, 1225
 - precedence graph and, 816–817
 - in the real world, 824
 - snapshot isolation and, 875–879
 - topological sorting and, 817–818
 - transactions and, 812–819, 821–826
 - view, 818–819, 867–868
- serializable schedules, 811, 812**
- serializable snapshot isolation (SSI) protocol, 878**
- server-side scripting, 416–418**
- server systems, 962–970**
 - categorization of, 963–964
 - data-server, 963–964, 968–970
 - defined, 962
 - transaction-server, 963–968
 - tree-like, 977–978
- services, 994**
- service time, 1230**
- servlets, 411–421**
 - alternative server-side frameworks, 416–421
 - example of, 411–413
 - life cycle and, 415–416
 - server-side scripting and, 416–418
 - sessions and, 413–415
 - support for, 416
 - web application frameworks and, 418–419
- session window, 505**
- set autocommit off, 144**
- set clause, 113**
- set default, 150**
- set-difference operation, 55, 750**
- set null, 150**
- set operations**
 - except, 88–89
 - intersect, 54–55, 87–88, 750
 - nested subqueries and, 98–101
 - query optimization and, 764–765
 - query processing and, 720–722
 - set comparison and, 99–101
 - set difference, 55
 - union, 53–54, 86–87, 750
- set orientation, 1220–1221**
- set role, 172**
- set statement, 201, 209**
- set transaction isolation level serializable, 822**
- set types, 366, 367**
- shadow-copy scheme, 914**
- shadowing, 571**
- shadow paging, 914**
- SHA-256 hash function, 1260**
- sharding, 473, 475–476, 1275**
- shard key, 479**
- shared and intention-exclusive (SIX) mode, 855**
- shared-disk architecture, 979, 980, 984–985**
- shared locks, 825, 835, 854, 880, 1124**
- shared-memory architecture, 21–22, 979–984, 1061–1064**
- shared-mode locks, 835**
- shared-nothing architecture, 979, 980, 985–986, 1040–1041, 1061–1063**
- shared scans, 785–786**
- Sherpa/PNUTS, 477, 1024, 1026, 1028–1030**
- show warnings, 746**
- shrinking phase, 841, 843**
- shuffle step, 486, 1061**
- SIMD (Single Instruction Multiple Data), 1064**
- simple attributes, 250, 265**
- simulation model, 1230**
- single inheritance, 275**
- Single Instruction Multiple Data (SIMD), 1064**
- single lock-manager, 1111**
- single sign-on system, 442–443**
- single-user systems, 962**
- single-valued attributes, 251**
- sites, 986**
- SIX (shared and intention-exclusive) mode, 855**
- skew**
 - aggregation and, 1049–1050
 - attribute-value, 1008
 - data distribution, 1008
 - in distribution of records, 660
 - execution, 1007, 1008, 1043
 - hash indices and, 1194
 - in joins, 1047–1048
 - parallel databases and, 974, 1007–1013, 1043, 1062
 - partitioning and, 715, 1007–1013
 - response time and, 1066
 - write skew, 876–877
- slicing, 530**
- sliding window, 505**
- slotted-page structure, 593**
- small-computer-system interconnect (SCSI), 562, 563**
- smart cards, 451, 451n9**

- smart contracts, 1258, 1269–1273**
- snapshot isolation**
 - concurrency control and, 872–879, 882, 916, 1131–1132
 - distributed, 1131–1132
 - performance tuning and, 1225
 - recovery systems and, 916
 - serializability and, 875–879
 - transactions and, 825–826, 1136
 - validation and, 874–875
- snowflake schema, 524**
- social-networking sites**
 - Big Data and, 467, 470
 - database applications for, 2, 3, 27
 - key-value store and, 471
 - streaming data and, 502
- soft deadlines, 894**
- soft forks, 1257, 1258**
- software-as-a-service model, 993**
- software RAID, 574, 575**
- solid-state drives (SSDs), 18, 560, 568–570, 693n3, 1229**
- some construct, 100, 100n10**
- some function, 96**
- sophisticated users, 24**
- sorting**
 - cost analysis of, 702–704
 - duplicate elimination and, 719–720
 - external sort-merge algorithm, 701–704
 - parallel external sort-merge, 1042–1043
 - query processing and, 701–704
 - range-partitioning, 1041–1042
 - topological, 817–818
- sort-merge-join algorithm, 708–712**
- sound axioms, 321**
- source-driven architecture, 522**
- space overhead, 624, 627–628, 634, 1202**
- Spark, 495–500, 508, 511, 1061**
- SPARQL, 375–376**
- sparse column data representation, 366**
- sparse indices, 626–632**
- spatial data**
 - design databases, 390–391
 - geographic, 387, 390–393
 - geometric, 388–390
 - indexing of, 672–675, 1186–1190
 - joins over, 719
 - quadtrees, 392, 674, 1186–1187
 - queries and, 393–394
 - R-trees, 663, 670, 674–676, 1187–1190
 - triangulation and, 388, 393
- spatial data indices, 672–675**
- spatial graph queries, 394**
- spatial graphs, 390**
- spatial-integrity constraints, 391**
- spatial join, 394**
- spatial networks, 390**
- specialization**
 - attribute inheritance and, 274–275
 - constraints on, 275–276
 - disjoint, 272, 275
 - entity-relationship (E-R) model and, 271–273
 - overlapping, 272, 275
 - partial, 275
 - single entity set and, 274
 - superclass-subclass relationship and, 272
 - total, 275
- specification of functional requirements, 17–18, 242**
- speedup, 972–974**
- splitting nodes, 641–645, 886**
- SQL Access Group, 1238**
- SQLAlchemy, 382**
- SQL/DS, 26**
- SQL environment, 163**
- SQL injection, 189, 438–439**
- SQLite, 668**
- SQLLoader, 1222**
- SQL MED, 1077**
- sql security invoker, 170**
- sqlstate, 205**
- SQL (Structured Query Language), 65–114**
 - advanced (*see* advanced SQL)
 - aggregate functions and, 91–96
 - application-level authorization and, 443–445
 - application programs and, 16–17
 - attribute specification in select clause, 83
 - authorization and, 66
 - basic types supported by, 67–68
 - blobs and, 156, 193, 594, 652
 - bulk loads and, 1221–1223
 - clobs and, 156, 193, 594, 652
 - create table and, 68–71
 - database modification and, 108–114
 - data definition for university databases, 69–71, 1288–1292
 - DDL and, 14–15, 65–71
 - decision-support systems and, 521
 - deletion and, 108–110
 - DML and, 16, 66
 - dumping and, 931
 - dynamic, 66, 184, 201
 - embedded, 66, 184, 197–198, 965, 1269
 - index creation and, 164–165, 664–665
 - injection and, 189, 438–439
 - inputs and outputs in, 747
 - insertion and, 110–111
 - integrity constraints and, 14–15, 66, 145–153
 - intermediate (*see* intermediate SQL)
 - isolation levels and, 821–826
 - JSON and, 369–370
 - limitations of, 468, 472
 - on MapReduce, 493–494
 - MySQL (*see* MySQL)
 - nested subqueries and, 98–107
 - nonstandard syntax and, 204

- NoSQL systems, 28, 473, 477, 1269, 1276
- null values and, 89–90
- OLAP in, 533–534, 536–538
- ordering display of tuples and, 83–84
- overview, 65–66
- PostgreSQL (*see* PostgreSQL)
- prepared statements and, 188–190
- prevalence of use, 13
- query processing and, 689–690, 701, 720
- query structure, 71–79
- relational algebra and, 80
- rename operation and, 79, 81–82
- ResultSet object and, 185, 187–188, 191–193, 638–639
- schemas and, 24, 66, 68–71
- security and, 438–439
- set operations and, 85–89
- standards for, 65, 1237–1238
- stream extensions to, 504–506
- string operations and, 82–83
- System R and, 26
- System R project and, 65
- theoretical basis of, 48
- transactions and, 66, 143–145, 965
- updates and, 111–114
- views and, 66, 137–143, 169–170
- where-clause predicates and, 84–85
- XML and, 372
- SSDs. *See* solid-state drives**
- SSI (serializable snapshot isolation) protocol, 878**
- stable storage, 804–805, 908–910**
- stale messages, 1149**
- stalls in processing, 733**
- standards**
 - ANSI, 65, 1237
 - anticipatory, 1237
 - CLI, 197, 1238–1239
 - database connectivity, 1238–1239
 - de facto, 1237
 - defined, 1237
 - formal, 1237
 - ISO, 65, 1237, 1241
 - object-oriented, 1239–1240
 - ODBC, 1238–1239
 - reactionary, 1237
 - SQL, 65, 1237–1238
 - X/Open XA, 1239
- star schema, 524–525**
- start-up costs, 974, 1066**
- start with/connect by prior syntax, 218**
- starved transactions, 841, 853**
- state-based blockchains, 1269, 1271**
- state machines, 1158–1161**
- Statement object, 186–187, 189**
- state of execution, 727**
- static hashing, 661, 1190–1195, 1202–1203**
- statistical analysis, 520, 527**
- statistics, 757–766**
 - catalog information and, 758–760
 - computing, 761
 - join size estimation and, 762–764
 - maintaining, 761
 - number of distinct values and, 765–766
 - selection size estimation and, 760, 762
- steal policy, 927**
- stepped-merge indices, 667, 1179–1181**
- stock market, streaming data and, 501**
- stop words, 385**
- storage, 587–617**
 - access time and, 561, 566, 567, 578
 - architecture for, 587–588
 - archival, 561
 - atomicity and, 804–805
 - authorization and, 19
 - backup (*see* backup)
 - Big Data and, 472–482, 668
 - bit-level striping and, 571–572
 - blockchain (*see* blockchain databases)
 - block-level striping and, 572
 - buffers and, 19, 604–610
 - byte amounts of, 18
 - checkpoints and, 920–922, 930
 - cloud-based, 28, 563, 992–993
 - column-oriented, 525–526, 588, 611–617, 734, 1182
 - crashes and, 607, 609–610
 - data access and, 910–912
 - data-dictionary, 602–604
 - data mining and, 27, 540–549
 - data-transfer rate and, 566, 569
 - in decision-support systems, 519–520
 - direct-access, 561
 - distributed (*see* distributed databases)
 - distributed file systems for, 472–475, 489, 1003, 1019–1022
 - dumping and, 930–931
 - durability and, 804–805
 - elasticity of, 1010
 - file manager for, 19
 - file organization and, 588–602
 - force output and, 912
 - geographically distributed, 1026–1027
 - hard disks for, 26
 - integrity manager and, 19
 - key-value, 471, 473, 476–480, 1003, 1023–1031
 - of large objects, 594–595
 - log disks, 610
 - in main-memory databases, 588, 615–617
 - mirroring and, 571–573, 576, 577
 - non-volatile, 560, 562, 587–588, 804, 908–910, 930–931
 - offline, 561
 - online, 561

- outsourcing, 28
- parallel (*see* parallel databases)
- physical (*see* physical storage systems)
- pointers and, 588, 591, 594–598, 601
- primary, 561
- punched cards for, 25
- random access, 567, 578
- recovery systems and, 908–912, 920–922, 930–931
- redundant arrays of independent disks, 562
- response time and, 572
- row-oriented, 611, 615
- R-trees and, 1189–1190
- scrubbing and, 575
- secondary, 561
- seek time and, 566, 566n2, 567, 692, 710
- sequential access, 561, 567, 578
- sharding and, 473, 475–476
- SQL DDL and, 67
- stable, 804–805, 908–910
- striping data and, 571–572
- structure and access-method definition, 24
- tertiary, 561
- transaction manager for, 19
- volatile, 560, 562, 804, 908
- wallets and, 450
- warehousing (*see* data warehousing)
- storage area network (SAN), 562, 563, 570, 934, 985**
- storage class memory (SCM), 569, 588, 948**
- storage manager, 18–20**
- store barrier, 983**
- stored functions/procedures, 1031**
- straggler nodes, 1060–1061**
- streaming data, 500–508**
 - algebraic operations and, 504, 506–508
 - applications of, 500–502
 - continuous, 731
 - defined, 500
 - fault tolerance with, 1074–1076
 - processing, 468, 1070–1076
 - querying, 502–506, 1070–1071
 - routing of tuples and, 1071–1073
- stream query languages, 503–506**
- strict two-phase locking protocol, 842, 843**
- string operations**
 - aggregate, 91
 - escape, 83
 - JDBC and, 184–193
 - like, 82–83
 - lower function, 82
 - query result retrieval and, 188
 - similar to, 83
 - trim, 82
 - upper function, 82
- stripe, 613–614**
- striping data, 571–572**
- strong entity sets, 259, 265–267**
- Structured Query Language. *See* SQL**
- structured types, 158–160**
- stylesheets, 408**
- subject of triples, 372**
- sublinear scaleup, 973**
- sublinear speedup, 972**
- subschemas, 12**
- suffix, 1243**
- sum function, 91, 139, 228, 536, 723, 766, 781**
- superclass-subclass relationship, 272, 274**
- superkeys, 43–44, 257–258, 309–310, 312**
- supersteps, 510**
- superusers, 166**
- supply chains, 1266, 1278**
- support, 547**
- Support Vector Machine (SVM), 544–545**
- swap space, 929**
- Sybase IQ, 615**
- Sybil attacks, 1255, 1256, 1264, 1266**
- symmetric fragment-and-replicate joins, 1046**
- symmetric-key encryption, 448**
- synchronous replication, 522–523, 1136**
- syntax, 199, 201–205, 689**
- sys.context function, 173**
- system architecture. *See* architecture**
- system catalogs, 602–604, 1009**
- system clock, 862**
- system error, 907**
- System R, 26, 65, 772–773, 772n3**
- table alias, 81**
- table functions, 200**
- table inheritance, 379–380**
- table partitioning, 601–602**
- tables**
 - defined, 1011
 - dimension, 524
 - dirty page, 941–947
 - distributed hash, 1013
 - fact, 524
 - foreign, 1077
 - partition, 1011–1014
 - pivot-table, 226–227, 528–529
 - in relational model, 9, 10, 37–40
 - in SQL DDL, 14–15
 - transition, 210
- tablets, 1011, 1025**
- tablet server, 1025**
- tab-separated values, 1222**
- tag library, 418**
- tags, 370–372, 406–407, 418, 440**
- tamper resistance, 1253–1255, 1259, 1260**
- tangles, 1278**
- tape storage, 561**
- Tapestry, 419**
- tasks, 1051–1052. *See also* workflow**
- Tcl, 206**
- telecommunications, database applications for, 3**
- temporal data, 347–351, 347n10**

- temporal data indices, 675–676
- temporal validity, 157
- term frequency (TF), 384
- termination of transactions, 806
- terms, 384, 1149
- ternary relationship sets, 249, 250, 284
- tertiary storage, 561
- test-and-set, 966
- test suites, 1234–1235
- text mining, 549
- textual data, 382–387. *See also*
 - information retrieval
 - keyword queries, 383, 386–387
 - overview, 382
 - relevance ranking and, 383–386
- Tez, 495
- TF-IDF approach, 384–385
- TF (term frequency), 384
- then clause, 212
- theta-join operations, 748–749
- third normal form (3NF), 317–319, 333–335
- Thomas' write rule, 864–866
- threads, 965, 982, 1062
- 3D-XPoint memory technology, 569
- 3NF decomposition algorithm, 334–335
- 3NF synthesis algorithm, 335
- 3NF (third normal form), 317–319, 333–335
- three-phase commit (3PC) protocol, 1107
- three-tier architecture, 23
- throughput
 - application design and, 1230–1232, 1234
 - in blockchain databases, 1274
 - harmonic mean of, 1231
 - improved, 808
 - parallel databases and, 971
 - range partitioning and, 1007
 - storage and, 572
 - system architectures and, 963, 971
 - transactions and, 808
- throughput test, 1234
- tickets and ticketing, 1133, 1279
- tiles, 392
- time intervals, 675–676
- time-lock transactions, 1273
- timestamps
 - concurrency control and, 861–866, 882
 - for data-storage systems, 480
 - defined, 154
 - distributed databases and, 1116–1118
 - generation of, 1117–1118
 - invalidation, 873n3
 - logical clock, 1118
 - logical counter and, 862
 - multiversion schemes and, 870–871
 - nondeterministic, 508n3
 - ordering scheme and, 862–864, 870–871, 1118
 - rollback and, 862–865
 - snapshot isolation and, 873, 873n2
 - system clock and, 862
 - Thomas' write rule and, 864–866
 - transactions and, 825
 - tuples and, 502, 503, 505
- time to completion, 1231
- timezone, 154
- TIN (triangulated irregular network), 393
- tokens, 1272
- Tomcat Server, 416
- top-down design, 273
- topic-partition system, 1073
- top-*K* optimization, 784
- topographical information, 393
- topological sorting, 817–818
- toss-immediate strategy, 608
- total failure, 909
- total generalization, 275
- total rollback, 853
- total specialization, 275
- TPC (Transaction Processing Performance Council), 1232–1234
- TPS (transactions per second), 1232
- tracks, 564
- training instances, 541, 543, 546
- transaction control, 66
- transaction coordinators, 1099, 1104, 1106–1107
- transaction identifiers, 913
- transaction managers, 18–21, 1098–1099
- Transaction Processing Performance Council (TPC), 1232–1234
- transactions, 799–828
 - aborted, 805–807, 819–820
 - actions following crashes, 923–925
 - active, 806
 - aggregation of, 1278
 - alternative models of processing, 1108–1110
 - association rules and, 546–547
 - atomicity of, 20–21, 144, 481, 800–807, 819–821
 - begin/end operations and, 799
 - blockchain, 1261–1263, 1268–1271, 1273
 - cascadeless schedules and, 820–821
 - check constraints and, 800
 - commit protocols and, 1100–1110
 - committed (*see* committed transactions)
 - commit work and, 143–145
 - compensating, 805
 - concept of, 799–801
 - concurrency control and (*see* concurrency control)
 - concurrent, 1224–1227
 - consistency of, 20, 800, 802, 807–808, 821–823
 - crashes and, 800
 - cross-chain, 1273
 - data mining, 540–549
 - defined, 20, 799
 - distributed, 989–990, 1098–1100
 - double-spend, 1261–1262, 1264
 - durability of, 20–21, 800–807

- failure of, 806, 907, 909, 1100
- force/no-force policy and, 927
- gas concept for, 1270–1271
- global, 988, 1098, 1132
- in-doubt, 1105
- integrity constraint violation and, 151–152
- isolation of, 800–804, 807–812, 819–826
- killing, 807
- local, 988, 1098, 1132
- locks and (*see* locks)
- log of (*see* log records)
- long-duration, 890–891
- minibatch, 1227
- multiversion schemes and, 869–872, 1129–1131
- observable external writes and, 807
- off-chain, 1275
- online, 4, 521
- performance tuning of, 1224–1227
- persistent messaging and, 990, 1016, 1108–1110, 1137
- read-only, 871
- real-time systems, 894
- recoverable schedules and, 819–820
- recovery systems and, 21, 803, 805
- remote backup systems and, 931–935
- restarting, 807
- rollback and, 143–145, 193, 196, 805–806, 922, 937–939, 945–946
- scalability and, 471
- serializability and, 812–819, 821–826
- shadow-copy scheme and, 914
- simple model for, 801–804
- as SQL statements, 826–828
- starved, 841, 853
- states of, 805–807
- steal/no-steal policy and, 927
- storage structure and, 804–805
- support for, 1028–1030
- terminated, 806
- time-lock, 1273
- timestamps and, 861–866
- two-phase commit protocol and, 989, 1016, 1276
- as unit of program execution, 799
- update, 871
- validation and, 866–869
- wait-for graph and, 851–852, 1113–1114
- write-ahead logging rule and, 926–929
- write operations and, 826
- transaction scaleup, 973**
- transactions-consistent snapshot, 1136**
- transaction-server systems, 963–968**
- transactions per second (TPS), 1232**
- transaction time, 347n10**
- TransactSQL, 199**
- transfer of control, 932–933**
- transformations**
 - data warehousing and, 523
 - equivalence rules and, 747–752
 - examples of, 752–754
 - join ordering and, 754–755
 - query optimization and, 747–757
 - relational algebra and, 747–757
- transition tables, 210**
- transition variables, 207**
- transitive closure, 214–216**
- transitive dependencies, 317n9, 356**
- transitivity rule, 321**
- translation, query processing and, 689–690**
- translation table, 568**
- tree-like server systems, 977–978**
- tree-like topology, 977**
- tree protocol, 846–848**
- trees**
 - B (*see* B-trees)
 - B⁺ (*see* B⁺-trees)
 - B-link, 886
 - decision-tree classifiers, 542
 - directory information, 1242, 1243
 - disjoint subtrees, 847
 - Generalized Search Tree, 670
 - k-d, 673–674
 - k-d B, 674
 - left-deep join, 773
 - LSM, 666–668, 1028, 1176–1182, 1215
 - Merkle, 1143–1146, 1268, 1269
 - Merkle-Patricia, 1269, 1275
 - multiple granularity and, 853–857
 - operator, 724, 1040
 - quadratic split heuristic and, 1189
 - quadtrees, 392, 674, 1186–1187
 - R, 663, 670, 674–676, 1187–1190
- tree topology, 977**
- triangulated irregular network (TIN), 393**
- triangulation, 388, 393**
- triggers**
 - alter, 210
 - defined, 206
 - disable, 210
 - drop, 210
 - need for, 206–207
 - nonstandard syntax and, 212
 - recovery and, 212–213
 - in SQL, 207–210
 - transition tables and, 210
 - when not to use, 210–213
- trim, 82**
- triple representation, 372–374**
- trivial functional dependencies, 311**
- true predicate, 76**
- true values, 96, 101**
- try-with-resources construct, 187, 187n3**
- tumbling window, 505**
- tuning. *See* performance tuning**
- tuning wizards, 1215**
- tuple-generating dependencies, 337**
- tuples**

- aggregate functions and, 91–96
- in Cartesian-product operation, 51, 52
- defined, 39
- deletion and, 108–110, 613
- duplicate, 103
- eager generation of, 726, 727
- insertion and, 110–111
- join operation for, 52–53 (*see also joins*)
- lazy generation of, 727
- logical routing of, 506–507, 1071–1073
- ordering display of, 83–84
- physical routing of, 1072–1073
- pipelining and, 691–692, 724–731
- query optimization and (*see query optimization*)
- query processing and (*see query processing*)
- ranking and, 219–223
- reconstruction costs, 612–613
- relational algebra and, 747–757
- in relational model, 39, 41, 43–46
- select operation for, 49
- set operations and, 54–55, 85–89
- streaming data and, 501–503, 505–507, 1071–1073
- timestamps and, 502, 503, 505
- updates and, 111–114, 613
- views and, 137–143
- windowing and, 223–226
- tuple variables, 81**
- Turing-complete languages, 1258, 1269, 1270**
- two-factor authentication, 441–442**
- 2NF (second normal form), 316n8, 341–342, 356**
- two-phase commit (2PC) protocol, 989, 1016, 1101–1107, 1161, 1276**
- two-phase locking protocol, 841–844, 871–872, 1129–1131**
- two-tier architecture, 23**
- type inheritance, 378–379**
- types**
 - blobs, 156, 193, 594, 652
 - clobs, 156, 193, 594, 652
 - complex (*see complex data types*)
 - large-object, 156, 158
 - performance tuning and, 1226
 - reference, 380–381
 - user-defined, 158–160, 378
- UML (Unified Modeling Language), 288–291**
- unary operations, 48**
- undo operation**
 - concurrency control and, 940–941
 - logical, 936–941
 - recovery systems and, 915–919, 922–925
 - rollback and, 916–919, 937–939
- undo pass, 944–946**
- undo phase, 923–925**
- Unified Modeling Language (UML), 288–291**
- uniform resource locators (URLs), 405–406**
- union all, 86, 97, 217n8**
- union of sets, 54, 750**
- union operation, 53–54, 86–87, 228**
- union rule, 321**
- unique construct, 103, 147**
- unique key values, 160–161**
- unique-role assumption, 345**
- uniquifiers, 649–650**
- United States**
 - address format used in, 250n1
 - identification numbers in, 447
 - primary keys in, 44–45
 - privacy laws in, 995
- universal front end, 404**
- Universal Serial Bus (USB) slots, 560**
- universal Turing machines, 16**
- university databases**
 - abstraction levels for, 11–12
 - application design and, 2–3, 5–7, 403–404, 431, 442–444
 - atomic domains and, 343
 - Big Data and, 499–500
 - blockchain, 1277
 - buffer-replacement strategies and, 607–609
 - Cartesian product and, 76–79
 - combination of schemas and, 270–271
 - complex attributes and, 249–252
 - concurrency control and, 879
 - consistency constraints for, 6, 13–15
 - decomposition and, 305–307, 310–312
 - deletion requests and, 109–110
 - design issues for, 346–347
 - distributed, 988
 - entities in, 243–246, 265–268, 281–283
 - entity-relationship diagram for, 263–264
 - full schema for, 1287–1288
 - functions and procedures for, 198–199
 - generalization and, 273–274
 - hash functions and, 1190–1193
 - incompleteness of, 243–244
 - indices and, 625–628, 664, 1017–1018
 - insertion requests and, 110–111
 - integrity constraints for, 145–153
 - mapping cardinalities and, 253–256
 - multivalued dependencies and, 336
 - query optimization and, 751–755, 775–778
 - query processing and, 690–691, 704, 723–724

- recursive queries and, 213–214, 217–218
- redundancy in, 243, 261–264, 269–270
- relational algebra for, 49–58
- relational model for, 9, 10, 37–47
- relational schema for, 41–43, 303–305
- relationship sets and, 246–249, 268–269, 282–283
- roles and authorizations for, 167–169
- sample data for, 1292–1298
- specialization and, 271–273
- SQL data definition for, 69–71, 1288–1292
- SQL queries for, 16
- storage and, 589–591, 597–601
- transactions and, 144, 826–827
- triggers and, 211–213
- triple representation of, 373–374
- unique key values for, 160–161
- user interfaces for, 24
- views and, 141–143
- University of California, Berkeley, 26**
- Unix, 83, 914**
- unknown values, 89–90, 96**
- unpartitioned site, 1056**
- unpin operations, 605**
- updatable result sets, 193**
- update-anywhere replication, 1137**
- update hot spots, 1225**
- updates**
 - authorization and, 14, 170, 171
 - batch, 1221
 - on B⁺-trees, 641–649
 - complexity of, 647–649
 - database modification and, 111–114
 - data warehousing and, 523
 - deletion time and, 641, 645–649
 - EXEC SQL and, 197
 - hashing and, 624, 1197–1202
 - inconsistent, 1140–1142
 - indices and, 630–632
 - insertion time and, 641–645, 647, 649
 - lazy propagation of, 1122, 1136
 - log records and, 913–914, 917–918
 - lost, 874
 - LSM trees and, 1178–1179
 - performance tuning and, 1221–1223, 1225–1227
 - privileges and, 166–167
 - query optimization and, 784–785
 - reconciliation of, 1142–1143
 - replication and, 1015–1016
 - shipping SQL statements to database and, 187
 - snapshot isolation and, 873–879
 - triggers and, 208, 212
 - tuples and, 111–114, 613
 - of views, 140–143
- update transactions, 871**
- upgrade, 843**
- URLs (uniform resource locators), 405–406**
- U.S. National Institute for Standards and Technology, 288**
- USB (Universal Serial Bus) slots, 560**
- user-defined types, 158–160, 378**
- user-interface layer, 429**
- user interfaces**
 - application architectures and, 429–434
 - application programs and, 403–405
 - back-end component of, 404
 - business-logic layer and, 430, 431
 - client-server architecture and, 404
 - client-side scripting and, 421–429
 - common gateway interface standard and, 409
 - cookies and, 410–415, 411n2, 439–440
 - CRUD, 419
 - data access layer and, 430–434
 - disconnected operation and, 427–428
 - front-end component of, 404
 - HTTP (*see* HyperText Transfer Protocol)
 - mobile application platforms and, 428–429
 - for naïve users, 24
 - presentation layer and, 429
 - responsive, 423
 - security and, 437–446
 - for sophisticated users, 24
 - storage and, 562–563
 - Web services (*see* World Wide Web)
 - web services and, 426–429
- user requirements in database design, 17–18, 241–242, 274**
- utilization of resources, 808**
- validation**
 - concurrency control and, 866–869, 882
 - distributed, 1119–1120
 - first committer wins and, 874
 - first updater wins and, 874–875
 - phases of, 866
 - recovery systems and, 916
 - snapshot isolation and, 874–875
 - test for, 868
 - view serializability and, 867–868
- validation phase, 866**
- valid interval, 870**
- valid time, 157, 347–350, 347n10**
- value for entity set attributes, 245**
- value set of attributes, 249**
- varchar, 67–68, 70**
- variable-length records, 592–594**

- variety of data, 468
- VBScript, 417
- vector data, 392–393
- vector processing, 612
- Vectorwise, 615
- velocity of data, 468
- verification of contents, 1145
- version numbering, 1141
- versions period for, 157
- version-vector scheme, 1141–1142
- Vertica, 615
- vertical partitioning, 1004
- view definition, 66
- view equivalence, 818, 818n4
- view level of abstraction, 10–12
- view maintenance, 140, 779–782, 1138–1140, 1215–1216
- views
 - authorization on, 169–170
 - with check option, 143
 - create view, 138–143, 162, 169
 - deferred maintenance and, 779, 1215–1216
 - defined, 137–138
 - deletion and, 142
 - immediate maintenance and, 779, 1215–1216
 - insertion and, 141–143
 - materialized (*see* materialized views)
 - performance tuning and, 1215–1216
 - SQL queries and, 138–139
 - update of, 140–143
- view serializability, 818–819, 867–868
- virtual machines (VMs), 970, 991–994
- virtual nodes, 1009–1010
- Virtual Private Database (VPD), 173, 444–445
- virtual processor approach, 1010n3
- Visual Basic, 184, 206
- visualization tools, 538–540
- VMs (virtual machines), 970, 991–994
- volatile storage, 560, 562, 804, 908
- volume of data, 468
- VPD (Virtual Private Database), 173, 444–445
- wait-die scheme, 850, 1112
- wait-for graphs, 851–852, 1113–1114
- WAL (write-ahead logging), 926–929, 934
- WANs (wide-area networks), 989
- weak entity sets, 259–260, 267–268
- wear leveling, 568
- web application frameworks, 418–419
- web-based services, database applications for, 3
- web crawlers, 383
- Weblogic Application Server, 416
- WebObjects, 419
- web servers, 408–411
- web services, 423–429
 - defined, 426
 - disconnected operation and, 427–428
 - interfacing with, 423–426
 - mobile application platforms, 428–429
- web sessions, 408–411
- WebSphere Application Server, 416
- when clause, 212
- when statement, 208
- where clause
 - aggregate functions and, 91–96
 - basic SQL queries and, 71–79
 - between comparison, 84
 - on multiple relations, 74–79
 - in multiset relational algebra, 97
 - not between comparison, 84
 - null values and, 89–90
 - predicates, 84–85
 - query optimization and, 774–777
 - ranking and, 222
 - rename operation and, 79, 81–82
- security and, 445
- set operations and, 85–89
- on single relation, 71–74
- string operations and, 82–83
- transactions and, 824, 826–827
- while loop, 196
- while statements, 201
- wide-area networks (WANs), 989
- wide column data representation, 366
- wide-column stores, 1023
- windows and windowing, 223–226, 502–506
- WiredTiger, 1028
- wireframe models, 390
- with check option, 143
- with clause, 105–106, 217
- with data clause, 162
- with grant option, 170–171
- with recursive clause, 217
- with timezone specification, 154
- witness data, 1258
- word count program, 483–486, 484n2, 490–492
- workers, 1051
- workflow
 - business-logic layer and, 431
 - database design and, 291–292
 - distributed transaction processing and, 1110
 - management systems for, 990
- workload, 783, 1215, 1217
- workload compression, 1217
- work stealing, 1048, 1062
- World Wide Web
 - application design and, 405–411
 - cookies and, 410–415, 411n2, 439–440
 - encryption and, 447–453
 - growth of, 467
 - HTML and (*see* HyperText Markup Language)
 - HTTP and (*see* HyperText Transfer Protocol)
 - impact on database systems, 27
 - security and, 437–446
 - servers and sessions, 408–411

- URLs and, 405–406
- WORM (write once, read-many)**
 - disks, 561, 1022
- wound-wait scheme, 850
- wrappers, 1077, 1236
- write-ahead logging (WAL),
 - 926–929, 934
- write amplification, 1180
- write once, read-many (WORM)
 - disks, 561, 1022
- write operations, 826
- write-optimized index structures,
 - 665–670
- write phase, 866
- write quorum, 1124
- write skew, 876–877
- write-write contention, 1225
- W-timestamp, 862, 865, 870
- X.500 directory access protocol,
 - 1241
- XML (Extensible Markup Language)
 - emergence of, 27
 - flexibility of, 367–368
 - as semi-structured data
 - model, 8, 27
 - SQL in support of, 372
 - tags and, 370–372
 - for transferring data, 423
- X/Open XA standards, 1239
- XOR operation, 448
- XPath, 372
- XQuery, 372
- XSRF (cross-site request forgery), 439–440
- XSS (cross-site scripting),
 - 439–440
- Yahoo Message Bus service,
 - 1137–1138
- Zab protocol, 1152
- ZooKeeper, 1150, 1152